



Tecnun
Universidad
de Navarra

ESCUELA DE INGENIERÍA
INGENIARITZA ESKOLA
SCHOOL OF ENGINEERING

Proyecto Fin de Grado

INGENIERIA EN ORGANIZACIÓN INDUSTRIAL

Desarrollo de un Sistema de Sentiment Analysis para la Predicción de Movimientos Bursátiles en el Sector Financiero

Ander Corta Arrieta

Donostia-San Sebastián, junio de 2025

INDICE DE CONTENIDOS

Contenido

RESUMEN	XII
ABSTRACT.....	XIII
1 INTRODUCCIÓN	1
1.1 Antecedentes	1
1.2 Objetivos	2
1.2.1 Recopilación, extracción y filtrado automático de noticias financieras	2
1.2.2 Análisis de sentimiento	3
1.2.3 Integración de información financiera	3
1.2.4 Análisis de modelo predictivo mediante métodos de machine learning	4
2 MARCO TEÓRICO Y ESTADO DEL ARTE	5
2.1 Introducción al mercado bursátil y su evolución histórica	5
2.2 Estrategias tradicionales de inversión	5
2.3 Introducción al análisis de sentimiento en finanzas	7
2.4 Técnicas fundamentales de procesamiento de lenguaje natural (NLP) ...	8
2.4.1 Tokenización.....	9
2.4.2 Lematización y Stemming.....	10
2.4.3 TF-IDF (Term Frequency-Inverse Document Frequency).....	10
2.4.4 Word embeddings.....	11
2.5 Modelos y técnicas de predicción de series temporales bursátiles	12
2.5.1 Random Forest.....	12

2.5.2	Modelos basados en redes neuronales: RNN, LSTM y GRU	13
2.6	Herramientas y librerías existentes para análisis de sentimiento y trabajar en python	18
3	DEFINICIÓN DEL PROYECTO	21
3.1	Formulación del problema.....	21
3.2	Hipótesis y objetivos experimentales	21
3.3	Criterios de evaluación y validación	22
3.4	Alcance y limitaciones.....	23
3.4.1	Alcance.....	23
3.4.2	Limitaciones.....	23
4	OBTENCIÓN Y TRATAMIENTO DE NOTICIAS	25
4.1	Recolección de noticias financieras (web scraping y APIs)	25
4.2	Limpieza y normalización de textos	27
4.3	Organización y almacenamiento de los datos.....	28
5	ANÁLISIS COMPARATIVO DE LIBRERÍAS DE SENTIMENT ANALYSIS Y SELECCIÓN	31
5.1	Librerías consideradas.....	31
5.2	Metodología para comparar rendimiento	31
5.2.1	Resultados: FinBERT vs opinión real	35
5.2.2	Resultados: VADER vs opinión real	37
5.3	Análisis y próximas interpretaciones:	39
5.3.1	FinBERT – Análisis de confianza media	39
5.3.2	VADER – Análisis de intensidad en errores	40
5.4	Conclusiones y selección.....	42

6	DESARROLLO DEL DATASET	43
6.1	Recopilación de noticias	44
6.2	Análisis de sentimiento	44
6.3	Agrupación diaria	44
6.4	Valores de bolsa	46
6.5	Dataset final	47
7	ANÁLISIS, ENTRENAMIENTO Y VALIDACIÓN DEL MODELO	49
7.1	Preparación de los datos: alineación de datos.....	49
7.2	División en conjunto de entrenamiento y prueba	50
7.3	Entrenamiento del modelo: Configuración y conjuntos de variables.	51
7.4	Ajuste de hiperparámetros	51
7.5	Evaluación de métricas	52
8	ANÁLISIS DE RESULTADOS Y DISCUSIÓN.....	53
8.1	Interpretación de los resultados obtenidos.....	53
8.2	Análisis de la correlación entre sentimiento y variación de precios	58
8.3	Limitaciones del estudio y posibles sesgos.....	59
8.4	Pruebas con diferente acción.....	59
9	CONCLUSIONES Y TRABAJO FUTURO	63
9.1	Conclusiones generales del proyecto	63
9.2	Valor añadido y aportaciones personales	63
9.3	Propuestas de mejora y líneas futuras de investigación	64
10	PRESUPUESTO.....	65
11	BIBLIOGRAFÍA.....	67
3.0	Estructura general del código	75

3.1 Web Scraping.....	75
3.2 Análisis de sentimientos.....	77
3.3 Agrupación de sentimientos	79
3.4 Construcción tabla financiera	81
3.5 Integración de los sentimientos.....	84
3.6 Modelo de predicción	87
3.7 Evaluación de semillas.....	92

INDICE DE FIGURAS

ILUSTRACIÓN 1: EJEMPLO DE TOKENIZACIÓN DE UN TEXTO EN INGLÉS	9
ILUSTRACIÓN 2: COMPARATIVA ENTRE LEMATIZACIÓN Y STEMMING	10
ILUSTRACIÓN 3: EJEMPLO GRÁFICO DE LA CLASIFICACIÓN Y CERCANÍA DE VECTORES Y PALABRAS	11
ILUSTRACIÓN 4: FUNCIONAMIENTO BÁSICO DE UN RANDOM FOREST	13
ILUSTRACIÓN 5: ILUSTRACIÓN DE UNA RED-NEURONAL	14
ILUSTRACIÓN 6: REDES NEURONALES PREALIMENTADAS VS RECURRENTE	16
ILUSTRACIÓN 7: ARQUITECTURA GENERAL DE UNA LSTM	17
ILUSTRACIÓN 8: COMPARATIVA ENTRE LAS DIFERENTES ARQUITECTURAS	18
ILUSTRACIÓN 9: ESTRUCTURA DE TITULARES POR CADA PÁGINA	26
ILUSTRACIÓN 10: EJEMPLO DE TABLA APPLE	28
ILUSTRACIÓN 11: FORMATO DE LA TABLA SAMPLE CON OPINIONREAL	32
ILUSTRACIÓN 12: EJEMPLO DE COLUMNAS AÑADIDAS POR FINBERT	32
ILUSTRACIÓN 13: EJEMPLO DE COLUMNAS AÑADIDAS POR VADER	33
ILUSTRACIÓN 14: FORMATO RESULTANTE DE LA TABLA FINAL	33
ILUSTRACIÓN 15: MATRIZ DE CONFUSIÓN DEL FINBERT	35
ILUSTRACIÓN 16: MATRIZ DE CONFUSIÓN DE VADER	37
ILUSTRACIÓN 17: ANÁLISIS DE FLUJO DE DATOS Y ESTRUCTURA DEL DATASET	43
ILUSTRACIÓN 18: DIVISIÓN DEL DATASET	50
ILUSTRACIÓN 19: COMPARACIÓN DEL RENDIMIENTO DEL MODELO A (SIN SENTIMIENTO)	53
ILUSTRACIÓN 20: COMPARACIÓN DEL RENDIMIENTO DEL MODELO B (CON SENTIMIENTO)	53
ILUSTRACIÓN 21: COMPARACIÓN TEST VALOR REAL VS PREDICCIÓN MODELO A (SIN SENTIMIENTO)	55
ILUSTRACIÓN 22: TEST VALOR REAL VS PREDICCIÓN MODELO B (CON SENTIMIENTO)	55
ILUSTRACIÓN 23: COMPARACIÓN ENTRE REAL VS NAIVE VS LSTM MODELO A	56
ILUSTRACIÓN 24: COMPARACIÓN ENTRE REAL VS NAIVE VS LSTM MODELO A	57
ILUSTRACIÓN 25: GRÁFICA DE CORRELACIÓN ENTRE VARIABLES	58
ILUSTRACIÓN 26: COMPARACIÓN TEST VALOR REAL VS PREDICCIÓN MODELO VS NAIVE A (SIN SENTIMIENTO) DE PFIZER	60
ILUSTRACIÓN 27: COMPARACIÓN TEST VALOR REAL VS PREDICCIÓN MODELO VS NAIVE B (SIN SENTIMIENTO) DE PFIZER	61

INDICE DE TABLAS

TABLA 1: KEYWORDS EMPLEADOS PARA EL FILTRADO Y BÚSQUEDA	27
TABLA 2: TABLA DE INDICADORES DE FINBERT	36
TABLA 3: TABLA DE INDICADORES DE VADER	38
TABLA 4: MATRIZ DE CONFUSIÓN PONDERADA DEL FINBERT (SCORE PROMEDIO)	39
TABLA 5: MATRIZ DE CONFUSIÓN PONDERADA VADER (SCORE PROMEDIO)	41
TABLA 6: COMPARACIÓN ENTRE MODELOS	54
TABLA 7: TABLA DE RENDIMIENTOS DE LOS MODELOS CON MAE NAIVE	57
TABLA 8: TABLA DEL RENDIMIENTO DEL MODELO PARA PFIZER	60
TABLA 9: COMPARACIÓN ENTRE DIFERENTES VALORES DE HIPERPARAMETROS	69
TABLA 10: COMPARACIÓN ENTRE SEMILLAS	70

RESUMEN

En el presente proyecto se desarrolla un sistema de recopilación, análisis y predicción del mercado bursátil, basado tanto en datos financieros como en variables sentimentales, con el objetivo de mejorar la capacidad de un modelo para predecir los movimientos del mercado. En primer lugar, se recopilan de forma automática noticias financieras relacionadas con una empresa en específico, empleando técnicas de *web scraping*. Se almacenan únicamente la fecha, el título, la fuente y el enlace a la noticia. El diseño de esta metodología se ha estructurado de forma modular para permitir realizar cambios de manera sencilla.

Posteriormente, se lleva a cabo un análisis de sentimiento utilizando modelos de procesamiento de lenguaje natural (PLN), lo cual permite transformar los titulares de las noticias en variables cuantitativas más fáciles de manejar. Toda esta información se integra en un conjunto de datos bursátiles del activo, complementado con medias móviles simples y exponenciales de distintos periodos de tiempo.

En la última fase del proyecto, se construyen dos modelos predictivos empleando una red neuronal LSTM. Se compara el rendimiento de la red utilizando, y sin utilizar, las variables sentimentales previamente generadas. La evaluación se realiza mediante métricas como MAE, R^2 y otras, permitiendo analizar el impacto de la variable de sentimiento en la capacidad predictiva del modelo.

ABSTRACT

This project develops a system for the collection, analysis, and prediction of the stock market, based on both financial data and sentiment variables, with the goal of improving a model's ability to predict market movements.

First, financial news related to a specific company is automatically collected using web scraping techniques. Only the date, title, source, and link to the news article are stored. The methodology is designed in a modular way to allow easy modifications.

Next, sentiment analysis is performed using natural language processing (NLP) models, which transform the news headlines into quantitative variables that are easier to manage. All this information is integrated into a dataset of the stock asset, supplemented with simple and exponential moving averages over different time periods.

In the final phase of the project, two predictive models are built using an LSTM neural network. The performance of the network is compared with and without the use of the previously generated sentiment variables. Evaluation is conducted using metrics such as MAE, R^2 , and others, allowing for an analysis of the impact of sentiment variables on the model's predictive ability.

1 INTRODUCCIÓN

Este Proyecto de Fin de Grado se enmarca en el ámbito del análisis de datos y noticias financieras, procesado de estos utilizando métodos de procesamiento de lenguaje natural (NLP) y la posterior implementación de un sistema para el análisis de sentimiento (Sentiment Analysis) aplicado al mercado bursátil. En la actualidad cientos de entidades compiten entre sí para encontrar una ventaja competitiva que les permita mejorar sus predicciones y, consecuentemente, su rentabilidad en el mercado de la inversión. En este contexto surge la idea de crear una herramienta automatizada que interprete cuál es el sentimiento del mercado para tratar de predecir de forma más precisa los futuros movimientos. En esta memoria se exponen todos los apartados del proceso que se han empleado para analizar la relación existente entre las noticias financieras y la evolución histórica del valor de las acciones de una empresa concreta.

1.1 Antecedentes

La creciente digitalización del sector financiero ha provocado un incremento significativo en la cantidad de información disponible en internet, lo que ha transformado profundamente las metodologías tradicionales de inversión y análisis de datos. En este nuevo contexto, han surgido e incorporado diversas herramientas basadas en la inteligencia artificial, destacando entre ellas el procesamiento de lenguaje natural (NLP), que se ha consolidado como una vía especialmente prometedora para comprender y anticipar el comportamiento del mercado (Daniel Lewington & Laura Sartenauer, 2022).

Históricamente, los inversores han recurrido a diversos métodos y criterios para apoyar la toma de decisiones financieras. Estos van desde indicadores fundamentales, como los balances contables y estados financieros de una empresa, hasta enfoques más complejos basados en patrones de comportamiento del precio, propios del análisis técnico (Melissa Horton, 2025).

Paralelamente, los medios de comunicación han desempeñado un papel clave como canal de difusión de información económica y financiera hacia el público general. No obstante, en determinadas circunstancias, estos medios no solo informan sobre acontecimientos bursátiles, sino que pueden influir activamente en ellos, generando cambios en el

sentimiento del mercado y, en consecuencia, afectando a la cotización de determinados activos (Talita Greyling & Stephanié Rossow, 2025).

Numerosos estudios y proyectos de investigación han abordado este tema, y en algunos casos con resultados exitosos, utilizando información procedente de fuentes tan diversas como Bloomberg, *The Wall Street Journal* o publicaciones en la red social anteriormente conocida como Twitter (ahora X). El uso de medios de comunicación tradicionales presenta la ventaja de ofrecer una información de mayor calidad, al estar elaborada por expertos con conocimientos específicos del mercado financiero. El potencial de establecer una relación sólida entre el contenido de las noticias y la evolución del valor bursátil de los activos es enorme y representa una vía de investigación de gran relevancia en el ámbito financiero actual (Talita Greyling & Stephanié Rossow, 2025).

1.2 Objetivos

Este proyecto contará con una serie de objetivos claramente definidos y una metodología concreta y estructurada, la cual se detalla a continuación:

1.2.1 Recopilación, extracción y filtrado automático de noticias financieras

La primera fase del proyecto consistirá en el desarrollo de un sistema automatizado que permita la recopilación y extracción de noticias históricas procedentes de una fuente previamente identificada y de reconocido prestigio en el ámbito financiero. El objetivo es reunir un volumen representativo de información sobre una empresa del mercado bursátil estadounidense, abarcando un periodo temporal superior a cinco años.

Para ello, se aplicarán técnicas de web scraping que permitirán extraer de forma estructurada los elementos más relevantes de cada noticia, tales como el título, la fecha de publicación, el autor (cuando esté disponible), el cuerpo del texto y otros metadatos pertinentes.

Una vez extraída la información, se realizará un proceso de filtrado de contenidos con el fin de conservar únicamente aquellas noticias que guarden una relación directa con las empresas analizadas o que contengan términos clave previamente definidos. Este filtrado

se basará en criterios semánticos y de relevancia contextual, con el objetivo de eliminar información redundante, poco significativa o no relacionada con el objeto de estudio.

Toda la información resultante será almacenada en una base de datos relacional diseñada para facilitar su posterior análisis, permitiendo acceder de forma eficiente a los contenidos y garantizando la integridad y trazabilidad de los datos recopilados.

1.2.2 Análisis de sentimiento

Una vez almacenada y filtrada toda la información relevante, se llevará a cabo el análisis de sentimiento de los textos recopilados. El objetivo de esta fase es identificar el tono general de cada noticia —positivo, negativo o neutral— así como otros posibles matices relacionados con la percepción del mercado.

Para ello, se utilizarán modelos de procesamiento de lenguaje natural (NLP) específicamente entrenados para tareas de análisis de sentimiento, tanto de propósito general como especializados en el ámbito financiero. Entre los modelos considerados se encuentran BERT, FinBERT, VADER o herramientas incluidas en bibliotecas como NLTK. La elección del modelo o combinación de modelos dependerá de su rendimiento en el dominio financiero y de su capacidad para captar con precisión el sentimiento expresado en las noticias (Karanikola et al., 2023).

1.2.3 Integración de información financiera

Una vez completado el análisis de sentimiento de cada noticia, se integrará esta información con los datos financieros correspondientes al valor bursátil de la empresa en la fecha de publicación. Se añadirán variables como el precio de apertura, cierre, máximo, mínimo diario y el volumen de transacciones. Para obtener estos datos se utilizarán librerías y APIs como Yahoo Finance, o en caso de necesitarlo, se utilizará una base de datos externa.

1.2.4 Análisis de modelo predictivo mediante métodos de machine learning

Como último paso, se procederá al desarrollo de un modelo predictivo basado en técnicas de machine learning. Para ello, se entrenará un modelo utilizando alguno de los algoritmos más comunes, como regresión logística, árboles de decisión, random forest o redes neuronales, combinando tanto variables financieras como datos derivados del análisis de sentimiento.

Una vez entrenado, el modelo será evaluado a través de métricas adecuadas (MAE, RMSE, R2, etc.) con el fin de comprobar su rendimiento. Finalmente, se analizarán los resultados obtenidos para valorar la utilidad real del sistema desarrollado y su capacidad para aportar valor en la predicción de movimientos bursátiles.

2 MARCO TEÓRICO Y ESTADO DEL ARTE

2.1 Introducción al mercado bursátil y su evolución histórica

Los comienzos del mercado bursátil se remontan a la Europa medieval, en un contexto en el que los comerciantes ya negociaban con activos y deudas en entornos relativamente organizados. Sin embargo, el término "bolsa" no vería su nacimiento hasta el siglo XVIII, en ciudades como Amberes y Brujas, donde los comerciantes se reunían para llevar a cabo transacciones mercantiles, monetarias o de deuda. A principios del siglo XVII surgió la primera empresa en emitir acciones al público: la Compañía Neerlandesa de las Indias Orientales (VOC). Esta innovación marcó el origen del primer mercado financiero formal con compraventa de acciones y sirvió de referencia para el desarrollo del sistema financiero moderno (Yolanda Moro, 2024).

La creación de este mercado de inversión en Ámsterdam atrajo a comerciantes de otras regiones del continente. Estos replicaron el modelo en sus países de origen, dando lugar a la aparición de bolsas de valores en ciudades como Londres (London Stock Exchange, siglo XVII), París y, posteriormente, a lo largo de los siglos XIX y XX, en otras ciudades como Nueva York, Tokio y Shanghái. A lo largo de la historia ha quedado demostrado que la existencia de estos mercados financieros facilita la liquidez y la transparencia de las inversiones, permitiendo transacciones más eficientes tanto para individuos como para entidades consolidadas (Yolanda Moro, 2024).

2.2 Estrategias tradicionales de inversión

Históricamente, han existido muchas formas diferentes de utilizar el dinero para generar rentabilidad. **La Especulación y la inversión**, son dos de los conceptos más conocidos dentro del ámbito financiero, y también dos de los más fundamentales. La diferencia principal entre ambos radica tanto en el horizonte temporal en el que operan como en el nivel de riesgo que implican, aunque siguen existiendo excepciones a esta norma.

La *inversión* se plantea como una estrategia orientada a obtener retornos más seguros, normalmente a través de intereses o dividendos. Por otro lado, la *especulación* suele implicar un mayor nivel de riesgo y se centra en la compraventa de activos en un periodo de tiempo más corto. Mientras que la inversión busca una fuente estable de ingresos a largo plazo, la especulación persigue rendimientos anormalmente altos asumiendo un riesgo considerablemente mayor. (Joseph Nguyen, 2024).

Hay distintas metodologías a utilizar en el análisis y valoración de activos financieros, cada una correspondiente a ciertas herramientas y enfoques propios. Las más populares son el *análisis fundamental* y el *análisis técnico*, ambas proporcionando perspectivas complementarias acerca del movimiento del mercado.

El análisis fundamental busca calcular el valor intrínseco de una acción mediante el estudio de los estados financieros de la empresa, como el balance, la cuenta de resultados y el flujo de caja. A través de este enfoque, se realiza una valoración integral de la empresa, con el objetivo de estimar si el valor de mercado del activo se corresponde con su valor real o estimado. Esta metodología permite evaluar si una acción está sobrevalorada o infravalorada, al comparar su precio actual en el mercado con el valor calculado. A diferencia del análisis técnico, el análisis fundamental se centra en factores económicos, financieros y cualitativos de la empresa, sin basarse en gráficos ni en tendencias de mercado a corto plazo. (Elisabet Furió, 2016).

El análisis técnico, por su parte, se basa en el estudio de los movimientos históricos del precio y del volumen de los activos, utilizando gráficos y herramientas estadísticas para identificar patrones y prever posibles tendencias futuras. A diferencia del análisis fundamental, el análisis técnico no considera los datos financieros de la empresa, sino que se centra exclusivamente en el comportamiento del mercado. Aunque puede aplicarse de forma independiente, en muchos casos los inversores combinan ambos enfoques para tomar decisiones más fundamentadas (Elisabet Furió, 2016).

Dentro del ámbito de las finanzas, se han propuesto diferentes teorías para explicar cómo se determinan los precios de los activos en los mercados. Una de las más influyentes es la **Hipótesis de los Mercados Eficientes (EMH, por sus siglas en inglés)**.

Esta hipótesis sostiene que toda la información disponible —ya sea pública como los informes financieros, o incluso rumores y expectativas que circulan entre los inversores— se refleja de forma inmediata en los precios de las acciones y demás activos.

Es decir, el mercado “procesa” la información de manera tan eficiente, que los precios actuales ya incorporan todo lo que se sabe en ese momento (José Antonio González, 2018).

En consecuencia, la EMH afirma que resulta virtualmente imposible identificar consistentemente valores infravalorados o supervalorados con el fin de lograr beneficio a corto plazo por medio de la compraventa. Además, según esta teoría, ningún análisis, no importa lo sofisticado que sea, puede superar consistentemente el rendimiento medio del mercado sin estar tomando un nivel más alto de riesgo (José Antonio González, 2018).

El trading algorítmico está estrechamente relacionado con la automatización de las transacciones en los mercados financieros, utilizando algoritmos informáticos complejos para ejecutar estrategias de inversión con rapidez y precisión, aprovechando oportunidades que un ser humano no podría detectar. De esta forma, se pueden alcanzar rendimientos más elevados, lo que justifica su amplio uso a nivel mundial, con una tasa de crecimiento esperada del 10% en los próximos ocho años (IMARC, 2025).

2.3 Introducción al análisis de sentimiento en finanzas

La proliferación de las redes sociales y de internet en su conjunto ha tenido como consecuencia un aumento en la cantidad de información disponible. En esta tesitura, surgen técnicas como el análisis de sentimiento, donde se analizan grandes volúmenes de texto y se clasifican en categorías como positivo, negativo o neutral. Las fuentes de información para estos métodos son muy diversas; pueden obtenerse a partir de artículos de internet, reseñas en línea, encuestas y demás. Todo esto puede ayudar a las empresas a mejorar la atención al cliente, la calidad de sus servicios o la toma de decisiones.

La importancia de implementar esta tecnología reside en la ventaja competitiva que se puede alcanzar. Los análisis de sentimiento ayudan a las compañías a filtrar opiniones y detectar sesgos personales, con el fin de lograr un punto de vista lo más objetivo posible sobre lo que se debe mejorar dentro de la empresa o del producto, por ejemplo. Además, el análisis de sentimiento permite a las compañías analizar y extraer información de forma rápida y comprensible (IBM, 2025).

Si se analiza el uso real de estas tecnologías en el mercado financiero, se concluye que el análisis de sentimiento del mercado examina la percepción general de los inversores respecto a uno o varios activos, lo cual, consecuentemente, puede llegar a impactar en el

precio de dichos activos, propiciando movimientos. Utilizando el análisis de sentimiento como herramienta complementaria, un inversor puede tomar decisiones mejor razonadas.

En el mercado ya existen ejemplos concretos de cómo los análisis de sentimiento han mejorado distintas áreas dentro de diversas empresas. Tomando como ejemplo a Coca-Cola, esta compañía lleva décadas siendo un referente en el sector de las bebidas azucaradas. Coca-Cola comenzó a utilizar el análisis de sentimiento con el objetivo de preservar su reputación. La empresa analiza menciones en redes sociales, reseñas y comentarios de sus clientes para detectar fácilmente el origen de las críticas, identificar las áreas que requieren mayor atención y así mantener una percepción positiva por parte del público (Faster Capital, 2025).

Un ejemplo relevante de la aplicación práctica del análisis de sentimiento en entornos financieros se encuentra en un estudio reciente centrado en el índice DAX40. En dicho trabajo, se construyó un índice impulsado por análisis de sentimiento a partir de noticias en alemán e inglés sobre las empresas que componen este índice bursátil. A diferencia de los productos tradicionales que ajustan sus ponderaciones en fechas predefinidas, esta propuesta permitió realizar ajustes dinámicos en función del sentimiento extraído de la información disponible en línea. Los resultados obtenidos muestran una rentabilidad anualizada del 7,51% durante un periodo de casi seis años, en contraste con el 2,13% registrado por el DAX40, incluso considerando los costes de transacción. Este caso pone de manifiesto el potencial del análisis de sentimiento como herramienta eficaz para mejorar el rendimiento de estrategias de inversión pasiva (Billert & Conrad, 2024).

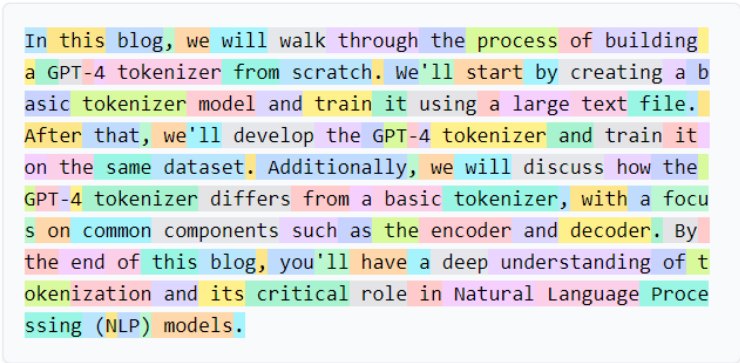
2.4 Técnicas fundamentales de procesamiento de lenguaje natural (NLP)

El Procesamiento de Lenguaje Natural (NLP, por sus siglas en inglés) es una de las ramas de la inteligencia artificial que se enfoca en la interacción entre las computadoras y el lenguaje humano. El NLP permite a los ordenadores interpretar el lenguaje humano en forma de texto. Esta tecnología es útil tanto para automatizar tareas que antes requerían intervención humana, como para facilitar el análisis de datos, ya que es capaz de extraer grandes volúmenes de información.

En la actualidad, existen muchos modelos y técnicas para el NLP, cada uno centrado en ámbitos específicos como el análisis de sentimientos, la reducción de ruido o la mejora de la eficiencia computacional (Pablo Huet, 2024).

2.4.1 Tokenización

Este proceso consiste en segmentar un texto en componentes más fáciles de manejar para el algoritmo de NLP. Cada uno de estos *tokens* representa la unidad mínima que aún conserva un significado dentro del texto original.



```
In this blog, we will walk through the process of building  
a GPT-4 tokenizer from scratch. We'll start by creating a b  
asic tokenizer model and train it using a large text file.  
After that, we'll develop the GPT-4 tokenizer and train it  
on the same dataset. Additionally, we will discuss how the  
GPT-4 tokenizer differs from a basic tokenizer, with a focu  
s on common components such as the encoder and decoder. By  
the end of this blog, you'll have a deep understanding of t  
okenization and its critical role in Natural Language Proce  
ssing (NLP) models.
```

Ilustración 1: Ejemplo de tokenización de un texto en inglés

La tokenización cumple varias funciones dentro del procesamiento de textos, entre las que se encuentran el preprocesamiento del texto, simplificándolo para su posterior análisis, o la normalización del texto, lo cual permite aplicar posteriormente otros métodos como el *stemming*. En este contexto, existen varios tipos de tokenización: aquellas basadas en espacios, que utilizan los espacios en blanco para delimitar los límites entre tokens (útiles y muy sencillas en la mayoría de los casos); otras más complejas, basadas en la utilización de reglas predefinidas; y, por último, algunas que emplean el aprendizaje automático y modelos preentrenados para determinar los límites de los tokens de manera más precisa (Pablo Huet, 2024).

La tokenización tiene aplicaciones prácticas muy relevantes, como servir de antesala para el análisis de sentimientos, la clasificación de texto para asignar categorías o, por ejemplo, la traducción de información (Pablo Huet, 2024).

2.4.2 Lematización y Stemming

El stemming es una técnica que corta los sufijos de las palabras para convertirlas a su forma más básica. Palabras como "caminata", "camina" o "caminando" se convertirían todas en "camin". Esta técnica no necesariamente devuelve una palabra con significado y, además, ignora el contexto en el que se encuentran las palabras (Pablo Huet, 2024).

La lematización, por otro lado, es una técnica más compleja y precisa. Reduce las palabras a su forma canónica o raíz con significado. Por ejemplo, "estudiando", "estudiar" y "estudio" se agrupan todas como "estudiar". La lematización sí tiene en cuenta el contexto de las palabras, utilizando un diccionario y una comprensión gramatical preexistente (Pablo Huet, 2024).

Stemming vs Lemmatization

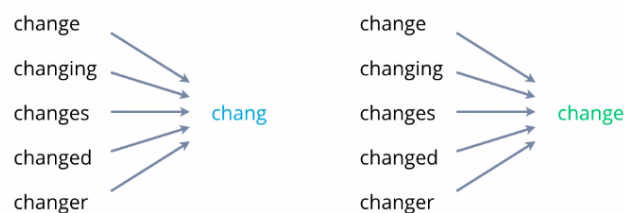


Ilustración 2: Comparativa entre Lematización y Stemming

Existen varios algoritmos como Snowball, Porter o aquellos que utilizan WordNet. Todos estos algoritmos tienen como objetivo diferentes fines, como mejorar la precisión de los motores de búsqueda, facilitar el análisis de sentimientos o la clasificación de texto (Pablo Huet, 2024).

2.4.3 TF-IDF (Term Frequency-Inverse Document Frequency)

El TF-IDF es una técnica utilizada en NLP para detectar qué palabras son las más importantes dentro de un texto, ayudando consecuentemente a la compresión y análisis de datos textuales (Pablo Huet, 2024).

Este método combina dos métricas diferentes: *Term Frequency* (TF) y *Inverse Document Frequency* (IDF). (Ver anexo 2)

2.5 Modelos y técnicas de predicción de series temporales bursátiles

El análisis de series temporales bursátiles se ha convertido, durante las últimas décadas, en una de las herramientas más potentes a la hora de tratar de anticipar movimientos en el mercado y, consecuentemente, facilitar la toma de decisiones para maximizar el rendimiento de las inversiones. Estas series representan, de forma cronológica, cómo han evolucionado factores relativos a la cotización bursátil de un activo: valor de cierre, volumen de transacciones, precio máximo, mínimo y demás. No son pocos los modelos que tratan de predecir el comportamiento futuro de estos activos; cada uno de ellos trabaja de forma diferente, con mayor o menor éxito.

2.5.1 Random Forest

El modelo de **Random Forest** es una técnica de *machine learning* basada en los árboles de decisión. Su funcionamiento se basa en la creación de múltiples árboles de decisión, donde las ramas representan diferentes decisiones o bifurcaciones, utilizando subconjuntos aleatorios de los datos de entrenamiento. El resultado final de este modelo se obtiene mediante votación, en el caso de los modelos de clasificación, o mediante el promedio, en los modelos de regresión. Este modelo se utiliza ampliamente por su versatilidad, su facilidad de interpretación y su capacidad para modelar relaciones no lineales, además de su eficacia en el manejo de una gran cantidad de variables (José Luis Ruiz Solivella, 2024).

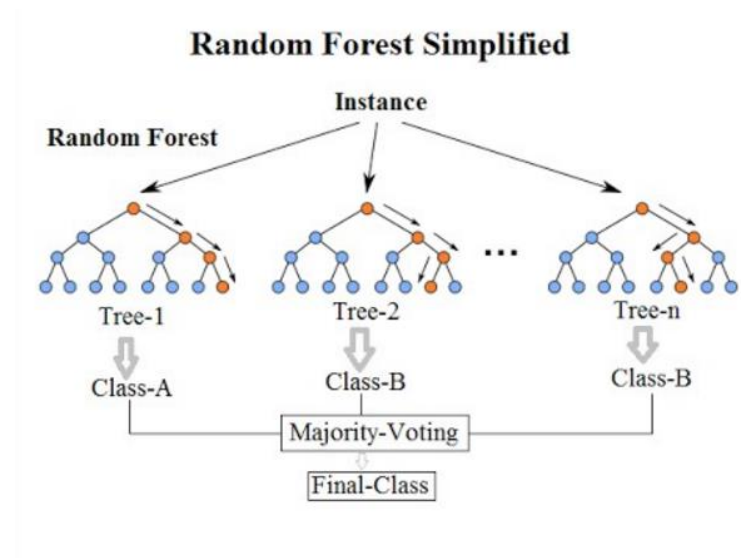


Ilustración 4: Funcionamiento básico de un Random Forest

Utilizando un ejemplo sencillo, si se quisiera predecir si el valor de una acción subirá o bajará, podrían considerarse algunas variables básicas: la variación del precio en el día anterior (subió/bajó), el volumen de transacciones (alto/bajo) y el sentimiento del mercado (positivo/negativo). A partir de estas variables, el modelo se plantearía preguntas como: **¿El sentimiento del mercado es positivo?** En caso de que la respuesta sea negativa, podría concluirse que la acción bajará. Si, por el contrario, el sentimiento es positivo, se continuarían realizando otras preguntas, como **¿El volumen de transacciones fue alto?**

Este proceso se repetiría con distintos formatos de árboles, generando múltiples combinaciones mediante el método de *Random Forest*, para determinar qué estructura es capaz de aproximarse mejor a la realidad o predecir con mayor precisión el comportamiento futuro del activo.

2.5.2 Modelos basados en redes neuronales: RNN, LSTM y GRU

2.5.2.1 Introducción a las redes neuronales

Una red neuronal artificial es un modelo de inteligencia artificial que simula, de forma simplificada, el funcionamiento del cerebro humano. Está compuesta por capas de nodos o "neuronas" artificiales distribuidas generalmente en tres niveles: una capa de entrada que

recibe los datos, una o varias capas ocultas que procesan la información, y una capa de salida que entrega el resultado. (IBM, 2025b)

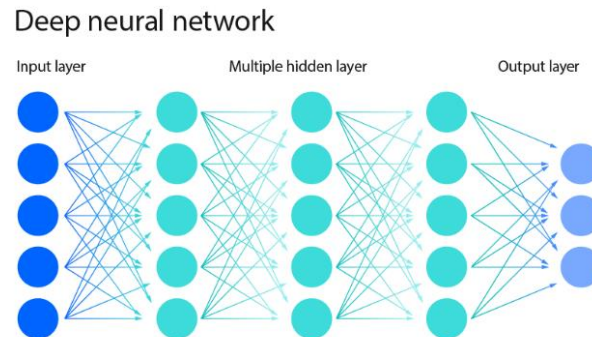


Ilustración 5: Ilustración de una red-neuronal

Cada nodo dentro de una red neuronal puede imaginarse como un pequeño modelo matemático, similar a una regresión lineal. Recibe una serie de datos de entrada, a cada uno de los cuales se le asigna una **ponderación** o peso que indica su importancia relativa. Estas ponderaciones no son arbitrarias: durante el entrenamiento, la red las ajusta para dar más relevancia a los factores que contribuyen positivamente a la predicción y restársela a los que no (IBM, 2025b).

El nodo calcula una **combinación lineal** de sus entradas multiplicando cada valor por su peso correspondiente, sumando luego un valor adicional llamado **sesgo** (bias).

$$Salida = \left(\sum_{i=0}^n W_i * X_i \right) + sesgo$$

Luego, esta salida se introduce en una **función de activación**, que transforma el resultado en una señal de salida, generalmente entre 0 y 1 o entre -1 y 1, dependiendo del tipo de activación utilizada (como sigmoide, ReLU, etc.). Si el valor obtenido supera un cierto umbral, el nodo se **activa**, lo que significa que transmite su señal a los nodos de la siguiente capa. De lo contrario, permanece inactivo. Este mecanismo de activación permite que la red filtre y seleccione qué información es relevante para cada etapa del procesamiento (IBM, 2025b).

Lo más importante de estas redes es su capacidad de aprender a partir de datos. Durante el entrenamiento, la red ajusta automáticamente los pesos de sus conexiones para mejorar sus predicciones. Este ajuste se realiza mediante un proceso llamado descenso del

gradiente, que permite reducir el error de sus resultados comparando las salidas obtenidas con las esperadas. A su vez, se emplea la retropropagación para calcular de forma precisa cómo cada peso contribuyó al error, y así realizar las correcciones necesarias. A través de esta repetición constante, la red va mejorando su capacidad de reconocer patrones y tomar decisiones con mayor precisión (IBM, 2025b).

Estas redes neuronales han demostrado ser muy eficaces en tareas como la clasificación de imágenes, el reconocimiento de voz o el análisis de datos estructurados. Sin embargo, tienen una limitación importante: no están diseñadas para recordar información previa. Esto dificulta su aplicación en problemas donde el orden de los datos o el contexto temporal son claves para una buena predicción (IBM, 2025b).

2.5.2.2 Redes neuronales recurrentes (RNN)

Las redes neuronales recurrentes (RNN) son una herramienta fundamental para el análisis de datos secuenciales, como las series temporales utilizadas en el ámbito financiero. A diferencia de las redes tradicionales, las RNN están diseñadas para recordar información de entradas anteriores, lo que les permite capturar la dinámica temporal presente en los datos. En el contexto bursátil, esto se traduce en la capacidad de modelar cómo los movimientos históricos del mercado influyen en los valores futuros de una acción (Cole Stryker, 2024).

Por ejemplo, si se introducen datos como precios de apertura y cierre, volumen de transacciones o indicadores técnicos (como la media móvil o el RSI), una RNN puede aprender patrones en la evolución temporal de un activo financiero. Además, este tipo de redes pueden integrarse con análisis de sentimiento obtenido a partir de noticias económicas o publicaciones en redes sociales, lo que permite crear modelos que consideren tanto los datos históricos como la percepción colectiva del entorno económico en tiempo real (Cole Stryker, 2024).

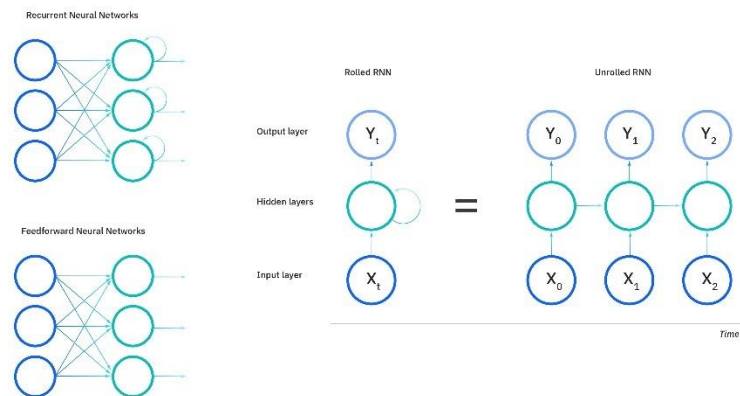


Ilustración 6: Redes neuronales prealimentadas vs recurrentes

Sin embargo, las RNN estándar presentan limitaciones significativas. Su capacidad para recordar información decrece con el tiempo, lo que dificulta la detección de dependencias a largo plazo. En finanzas, donde ciertos eventos pueden afectar el mercado días después de ocurrir, esta limitación reduce su efectividad. Además, durante el entrenamiento con grandes volúmenes de datos, las RNN pueden sufrir problemas de desvanecimiento o explosión del gradiente, lo que compromete la estabilidad y precisión del modelo (Cole Stryker, 2024).

2.5.2.3 Long Short Term Memory (LSTM)

Las redes **LSTM** son una extensión más sofisticada de las RNN, particularmente útiles en la predicción bursátil debido a su habilidad para capturar dependencias de largo plazo. A diferencia de las RNN tradicionales, las LSTM están diseñadas para conservar información relevante durante largos periodos, lo que es ideal para mercados financieros, donde ciertos patrones pueden desarrollarse lentamente o verse afectados por eventos pasados. Un ejemplo claro de esto serían las siguientes frases: “El banco central anunció una subida de los tipos de interés” y “Esto provocó una caída en los valores de las acciones”, si se utilizara una RNN convencional el modelo no podría interpretar la relación en caso de que las dos frases estuvieran muy lejos una de la otra (Cole Stryker, 2024).

Gracias a su estructura basada en una *celda de memoria* y en *puertas de control* (entrada, olvido y salida), las LSTM pueden seleccionar qué información guardar y cuál olvidar,

adaptándose dinámicamente a los datos que están procesando. Esto permite, por ejemplo, que el modelo recuerde una bajada en los tipos de interés semanas atrás que puede estar influyendo ahora en el comportamiento de un índice bursátil (Cole Stryker, 2024).

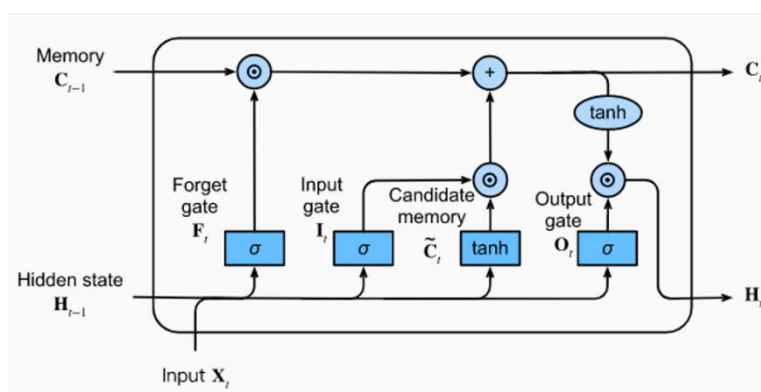


Ilustración 7: Arquitectura general de una LSTM

En combinación con análisis de sentimiento, las LSTM permiten integrar la evolución emocional del mercado —por ejemplo, el cambio de actitud frente a una empresa tras una noticia— con las tendencias de precios. También pueden emplearse para predecir eventos futuros como subidas o bajadas abruptas, basándose en patrones sutiles que ocurren en períodos extendidos (Cole Stryker, 2024).

Sin embargo, esta potencia tiene un coste: *las LSTM requieren más recursos computacionales y tiempo de entrenamiento que las RNN*, lo que puede ser un factor limitante cuando se trabaja con bases de datos bursátiles muy grandes o con sistemas de predicción en tiempo real (Cole Stryker, 2024).

2.5.2.4 Gated Recurrent Unit (GRU)

Las GRU ofrecen un equilibrio entre eficiencia computacional y capacidad de aprendizaje, lo que las convierte en una alternativa muy interesante para aplicaciones financieras donde se requiere tanto **rapidez como precisión**. Al igual que las LSTM, las GRU están diseñadas para aprender dependencias a largo plazo en datos secuenciales, pero utilizan una estructura más simple: dos puertas (de actualización y de reinicio) que controlan el flujo de información sin necesidad de una celda de memoria separada (Cole Stryker, 2024).

Esta eficiencia estructural hace que las GRU sean ideales para **sistemas de predicción bursátil en tiempo real**, donde el modelo necesita adaptarse rápidamente a nuevos datos del mercado, como noticias de última hora, movimientos políticos o anuncios económicos.

Además, su menor complejidad reduce la necesidad de grandes cantidades de datos o potencia de cálculo, lo que es útil cuando se entrena el modelo en dispositivos locales o entornos distribuidos (Cole Stryker, 2024).

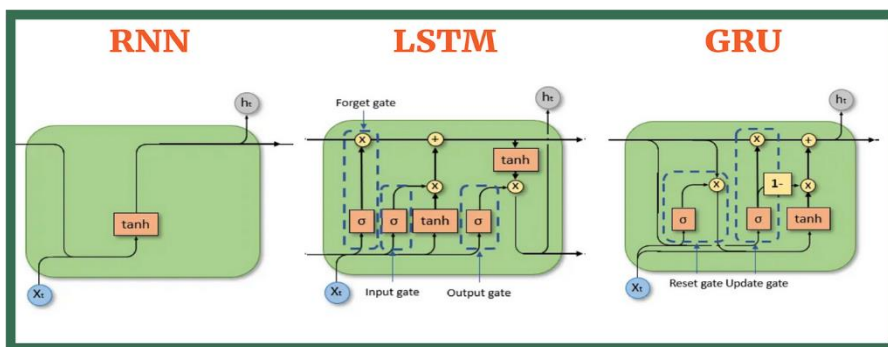


Ilustración 8: Comparativa entre las diferentes arquitecturas

En tareas específicas como la **clasificación del sentimiento financiero** (optimista vs. pesimista), la detección de eventos de alta volatilidad o la predicción del comportamiento de acciones con alta correlación, las GRU han demostrado ser igual de eficaces que las LSTM, pero con un menor coste computacional. Por ello, muchas plataformas financieras modernas integran arquitecturas basadas en GRU como parte de sus modelos de análisis de datos en tiempo real (Cole Stryker, 2024).

2.6 Herramientas y librerías existentes para análisis de sentimiento y trabajar en python

En el ámbito financiero, donde el lenguaje técnico y las expresiones propias del mercado son predominantes, se requieren modelos especializados. Uno de los más destacados es **FinBERT**, una versión adaptada del modelo BERT entrenada específicamente en textos financieros como informes, artículos y noticias económicas, capaz de clasificar el tono como positivo, negativo o neutral con alta precisión (Wiston Adrián Risso Charquero, 2023).

Asimismo, modelos más recientes como **BloombergGPT**, desarrollado por Bloomberg, han sido diseñados con grandes volúmenes de datos financieros para tareas avanzadas como análisis de sentimiento, generación de texto y comprensión contextual dentro del dominio económico (Bloomberg, 2023).

VADER es una herramienta de análisis de sentimiento basada en un enfoque léxico y de reglas, diseñada específicamente para captar con precisión las opiniones expresadas en medios digitales como redes sociales, comentarios en línea y foros. Utiliza un diccionario de palabras etiquetadas según su carga emocional, ya sea positiva o negativa, y combina estas etiquetas con una serie de reglas gramaticales y sintácticas para interpretar el tono general del texto (Skillcate AI, 2022).

3 DEFINICIÓN DEL PROYECTO

3.1 Formulación del problema

El mercado financiero y las noticias han estado históricamente muy relacionadas entre sí. Un escándalo en una empresa, la publicación de resultados económicos decepcionantes u otros eventos similares han tenido siempre un impacto directo en la confianza de los accionistas sobre el futuro de la compañía en la que han invertido su dinero. De la misma manera que las noticias financieras pueden afectar al valor de un activo, también los movimientos significativos en dicho activo suelen generar noticias, formando así un círculo vicioso en el que, en muchas ocasiones, resulta difícil distinguir la causa del efecto.

En esta tesitura, surge la siguiente cuestión:

¿Sería posible desarrollar un sistema automatizado que recopile noticias del sector financiero y las relacione con la evolución de un activo para predecir mejor su comportamiento?

A partir de esta pregunta, este Proyecto de Fin de Grado buscará dar una respuesta mediante la implementación de tecnologías actuales, con el objetivo de evaluar si existe una relación significativa entre el contenido de las noticias y el comportamiento del mercado bursátil.

3.2 Hipótesis y objetivos experimentales

Existe una relación comprobable y significativa entre las noticias del sector financiero y los movimientos en el precio y volumen de transacciones de las acciones de las empresas.

Partiendo de esta hipótesis, los principales objetivos de este proyecto serán los siguientes:

Diseño de un sistema automatizado que recopile y procese noticias procedentes de fuentes públicas relacionadas con un activo financiero concreto. Este sistema incluirá un mecanismo de filtrado que permita conservar únicamente aquellas noticias que resulten relevantes y potencialmente influyentes en el desempeño bursátil del activo en cuestión.

Implementación de técnicas de análisis de sentimiento mediante el uso de librerías existentes en Python. Estas herramientas permitirán clasificar automáticamente las noticias en categorías de sentimiento: positivo, negativo o neutral. Además, se evaluará el rendimiento de estas librerías utilizando métricas como precisión, recall y F1-score, para lo cual se seleccionará una muestra aleatoria, pero estadísticamente significativa, de noticias que serán clasificadas manualmente como referencia.

Vinculación temporal de la información textual con datos financieros históricos, de modo que cada noticia quede asociada a variables bursátiles como el precio de apertura, cierre, máximos y mínimos diarios, así como el volumen de transacciones correspondiente al mismo periodo temporal.

Tratamiento y estructuración de un dataset adecuado, el objetivo de este apartado será trabajar la información para construir una base de datos que contenga todas las variables relevantes necesarias para implementar posteriormente un modelo predictivo o para realizar un análisis de la relación entre las diferentes variables.

Implementación de un modelo predictivo: Como fase última del proyecto, se construye un modelo predictivo para predecir movimientos en el mercado bursátil mediante la fusión de variables financieras y datos obtenidos del análisis de sentimiento. Para ello se utilizan algoritmos de machine learning como la regresión logística, árboles de decisión, random forest, y redes neuronales recurrentes (RNN, LSTM, GRU), eligiendo al que resulte en el mejor equilibrio entre rendimiento, interpretabilidad y compatibilidad en contexto financiero.

3.3 Criterios de evaluación y validación

A la hora de evaluar la calidad y efectividad del sistema se utilizará una serie de criterios diferenciados:

Precisión de las librerías de análisis de sentimiento: Para medir este aspecto se emplearán métricas estándar como *accuracy*, *recall* y *F1-score*. Además, con el objetivo de visualizar los resultados de forma más clara, se utilizará también una matriz de confusión.

Desempeño del modelo predictivo: Se evaluará utilizando indicadores como el coeficiente de determinación (R^2) y el error cuadrático medio (*MSE*), que permitirán cuantificar el grado de ajuste del modelo y su capacidad para anticipar movimientos bursátiles.

3.4 Alcance y limitaciones

3.4.1 Alcance

El proyecto se centra exclusivamente en el análisis de sentimiento aplicado a noticias financieras relacionadas con una empresa en particular, aunque podrá emplearse otra empresa adicional en caso de ser necesario. Para realizar este proyecto se tomó concretamente una compañía estadounidense, durante un periodo aproximado de cinco años.

Para la clasificación de texto se emplean modelos de procesamiento de lenguaje natural (NLP) preentrenados. No se desarrollará un modelo propio desde cero, sino que se utilizarán librerías existentes ya entrenadas en contextos similares.

Los datos financieros se obtendrán a través de una de las siguientes dos vías: mediante el uso de APIs públicas y gratuitas, como la de Yahoo Finance, o bien a partir de conjuntos de datos (datasets) que contengan las cotizaciones bursátiles de la acción que se desea analizar.

Por último, se analizarán distintos modelos predictivos de machine learning, y se seleccionará aquel que ofrezca el mejor equilibrio entre rendimiento, interpretabilidad y adecuación a los objetivos del proyecto.

3.4.2 Limitaciones

Este proyecto contará con una serie de limitaciones, las cuales se presentarán a continuación:

Acceso restringido a medios relevantes: Fuentes clave como Bloomberg o The Wall Street Journal protegen su contenido mediante suscripciones, APIs cerradas o tecnologías anti-scraping. Esto reduce la cantidad y calidad de datos disponibles para el análisis.

Ambigüedad causal: Identificar una correlación entre el contenido de las noticias y el precio de las acciones no implica necesariamente causalidad. Las noticias pueden estar reflejando hechos ya ocurridos, lo que limita su valor predictivo.

Dependencia de modelos existentes: No es viable entrenar un modelo propio de análisis de sentimiento adaptado al dominio financiero específico del proyecto. Por tanto, se

depende completamente de herramientas externas, que pueden no ofrecer la precisión deseada.

Dificultad para captar matices complejos: Los modelos de sentimiento actuales tienen dificultades para interpretar ironía, lenguaje técnico o ambigüedades frecuentes en el periodismo financiero, lo que puede afectar negativamente la calidad del etiquetado.

Volumen de datos y filtrado incompleto: Debido a la elevada cantidad de noticias recopiladas, es imposible realizar un filtrado manual exhaustivo. Esto conlleva el riesgo de que se incluyan datos irrelevantes, ruidosos o poco representativos, lo cual puede distorsionar las conclusiones.

Limitación de contexto temporal y sectorial: El proyecto se focaliza en un número reducido de empresas y un sector específico. Los resultados pueden no ser extrapolables a otros sectores, geografías o periodos económicos.

Restricciones computacionales: El análisis de texto a gran escala y el entrenamiento de modelos complejos exigen recursos computacionales considerables. El proyecto opera dentro de limitaciones técnicas que pueden condicionar el volumen de datos procesado o el tipo de modelos utilizados.

4 OBTENCIÓN Y TRATAMIENTO DE NOTICIAS

4.1 Recolección de noticias financieras (web scraping y APIs)

La obtención de noticias constituye no solo uno de los apartados más importantes de todo el proyecto —ya que representa la base sobre la cual se construye el análisis posterior—, sino también una de las etapas con mayor dificultad técnica. En una primera fase, se exploraron diferentes alternativas para la recopilación de noticias: redes sociales, medios de comunicación tradicionales como Bloomberg, Yahoo Finance o Markets Insider, así como bases de datos ya existentes, como las disponibles en plataformas como Kaggle.

Sin embargo, la mayoría de los medios digitales presentan importantes obstáculos para su integración en un proyecto de este tipo. En primer lugar, muchas plataformas ofrecen APIs de pago con limitaciones en cuanto al volumen de solicitudes o el acceso histórico, lo que supone un coste económico considerable y una restricción funcional para un proyecto académico. Además, es frecuente que estos sitios implementen mecanismos anti-*scraping* que detectan y bloquean automáticamente cualquier intento automatizado de extracción de información. Otro problema adicional fue la escasa disponibilidad de archivos históricos de noticias sobre una empresa concreta, lo que dificultó la recopilación de información antigua necesaria para un análisis temporal significativo.

También se valoró brevemente el uso de **consultas automatizadas mediante Google Search**, pero esta opción fue descartada por una limitación crítica: el número de búsquedas diarias permitidas es extremadamente bajo para herramientas gratuitas, lo que hacía inviable cubrir el volumen de noticias necesarias para el estudio.

A pesar de estas dificultades, se optó por seguir adelante con la recopilación directa desde medios reconocidos, priorizando la calidad y fiabilidad de la fuente frente a la facilidad de acceso. En este sentido, se consideró que la información proveniente de medios oficiales y consolidados aporta mucho más valor que opiniones dispersas recogidas en redes sociales, cuya relevancia o veracidad resulta más difícil de validar.

HOME > NEWS

News for Apple Inc.

SEEKING ALPHA 16h

Apple seeks to use AI to boost iPhone battery life: report

TIPRANKS 17h

Apple iOS 19 update to use AI to improve battery life, Bloomberg says

TIPRANKS 17h

Apple iOS 19 update to use AI to improve battery life, Bloomberg says

TIPRANKS 17h

Apple Services 'more at-risk than ever before,' says Morgan Stanley

TIPRANKS 18h

Apple working on 'curved' iPhone, Bloomberg reports

TIPRANKS 18h

Mixed options sentiment in Apple (AAPL), with shares up \$11.56 (+5.83%) near \$209.83.

TIPRANKS 19h

Drink and Drug Firms Boost U.S. Manufacturing Health With \$1B Plus Investment Plans

TIPRANKS 19h

How NVDA, AAPL, and META Plan to Harness VR to Dominate the Future

TIPRANKS 20h

'We're Removing Alphabet Stock from Our Best Ideas List,' Says Wedbush

Ilustración 9: Estructura de titulares por cada página

La empresa elegida para el análisis fue **Apple Inc.**, debido a varios factores clave: se trata de una de las mayores compañías tecnológicas del mundo, con una extensa trayectoria bursátil, y ha sido durante muchos años la empresa con mayor capitalización del mercado. Este protagonismo mediático ha generado un volumen muy elevado de noticias asociadas a su nombre, lo que facilita considerablemente la recopilación de artículos históricos en comparación con otras empresas menos mediáticas.

El medio seleccionado como fuente de noticias fue **Business Insider**, por tratarse de una publicación económica de gran prestigio que, además, no impone restricciones severas para el proceso de *scraping*. Un aspecto especialmente positivo de esta fuente es la **facilidad de acceso a su archivo histórico**, ya que las noticias están organizadas en una estructura de paginación muy simple. El enlace al archivo de noticias de Apple sigue el formato sencillo en el que poder iterar y no cuenta con ningún sistema que dificulte la extracción de datos.

```
https://markets.businessinsider.com/news/aapl-stock?p={page}
```

Para llevar a cabo la recolección de datos se desarrolló un **script en Python** que automatiza el proceso de navegación y extracción de información en más de 200 páginas de resultados. Para su implementación, se emplearon las siguientes librerías:

- BeautifulSoup (para el análisis y parseo del contenido HTML)
- requests (para realizar las peticiones HTTP a las páginas web)
- pandas (para la organización de los datos en estructuras tipo DataFrame)
- pyodbc (para la conexión e inserción de datos en una base de datos Access)

- os (para manejar rutas de archivos y directorios del sistema)

El script extrae de forma sistemática los siguientes elementos de cada noticia:

- Fecha de publicación del artículo
- Título de la noticia
- Fuente del medio
- Enlace directo al artículo original

No se extrajo el cuerpo completo de las noticias debido a consideraciones de eficiencia, carga computacional y tiempo. Además, se asumió que en la mayoría de los casos el título ya proporciona una idea bastante clara del tono o intención del contenido, siendo suficiente para el análisis de sentimiento planteado en este proyecto.

4.2 Limpieza y normalización de textos

Para mejorar la calidad del análisis posterior, se implementó un sistema de filtrado que clasifica las noticias en tres grupos: **Apple**, **Entorno** y **Otros**. Esta clasificación se basa en la presencia de ciertas palabras clave (*keywords*) en el título de cada noticia.

<i>Categoría</i>	<i>Keywords</i>
Apple: Noticias relacionadas directamente con Apple	Apple, aapl, iphone
Entorno: Empresas tecnológicas y competidores	technology, tech, google, microsoft, samsung, amazon, intel, semiconductor, software

Tabla 1: Keywords empleados para el filtrado y búsqueda

Gracias a este filtrado, se facilita el trabajo posterior con los datos más relevantes para el análisis, permitiendo centrarse en aquellas noticias que tienen una mayor probabilidad de estar relacionadas con el comportamiento bursátil de la empresa.

Además, durante el procesamiento de los textos se han aplicado ciertas transformaciones básicas orientadas a estandarizar el contenido:

Conversión del texto a minúsculas, con el objetivo de garantizar una comparación uniforme entre títulos y palabras clave, independientemente del uso de mayúsculas o minúsculas en la redacción original.

Extracción directa del contenido sin etiquetas HTML, asegurando que los datos almacenados contengan únicamente el texto visible y legible por el usuario, libre de formatos o elementos estructurales propios del código fuente de la página web.

Aunque en esta versión se han recogido solo datos básicos para cada nota —título, fecha, fuente, y enlace—, el código se ha diseñado modular y escalable. De esta manera, se facilita su adaptación al llevarlo para otras empresas mediante cambios mínimos, nada más que reajustando el conjunto de palabras clave.

4.3 Organización y almacenamiento de los datos

Para estructurar correctamente la información y facilitar su tratamiento posterior, se utilizó una **base de datos relacional en Microsoft Access**. La elección de esta herramienta se debe principalmente a la familiaridad con su uso y a su sencillez, lo que la convierte en una opción práctica para el contexto del proyecto. Para gestionar esta base de datos desde Python, se empleó la librería **pyodbc**, que permite establecer una conexión directa con archivos .accdb, posibilitando la creación, modificación y manipulación de tablas de manera programática.



article_date	title	source	link
05/05/2025 12:08:01	Apple 10-Q highlights tariff mitigation tools, rising inventories, service driver	Seeking Alpha	https://seekingalpha.com/news/stocks/apple-10-q-highlights-tariff-mitigation-tools-rising-inventories-service-driver
05/05/2025 5:35:25	Microsoft (MSFT) Reclaims Top Spot from Apple with \$3.2T Market Cap After	TipRanks	/news/stocks/micro:
05/05/2025 0:41:22	Apple price target lowered to \$200 from \$235 at Phillip Securities	TipRanks	/news/stocks/apple-
05/04/2025 7:27:41	AAPL, AXP, BAC, KO, CVX: Buffett's Berkshire Bets Big on These 5 Stocks	TipRanks	/news/stocks/aapl-a
05/02/2025 20:46:42	Apple (AAPL) Partners with Anthropic to Develop AI-Powered Coding Platform	TipRanks	/news/stocks/apple-
05/02/2025 18:56:13	Analysts Divided on Apple (AAPL) as Tariff Fears Clash with Supply Chain Progress	TipRanks	/news/stocks/analys
05/02/2025 18:20:15	Apple teaming with Anthropic on AI software, Bloomberg reports	TipRanks	/news/stocks/apple-
05/02/2025 17:55:13	Apple teaming with Anthropic on AI software, Bloomberg reports	TipRanks	/news/stocks/apple-
05/02/2025 17:31:23	Apple Plans to Source 19 Billion Chips from the U.S.	TipRanks	/news/stocks/apple-
05/02/2025 16:35:45	Apple reports earnings beat, China weighs trade talks with U.S.: Morning Buz	TipRanks	/news/stocks/apple-
05/02/2025 16:33:26	SA analyst downgrades: AAPL, SMCI, MRNA, TRIP, IIPR, STLA, TWST	Seeking Alpha	https://seekingalpha

Ilustración 10: Ejemplo de tabla apple

Como se mencionó anteriormente, las noticias recopiladas se han agrupado según su relación con la empresa **Apple** o con su **entorno competitivo**. Aquellos artículos que no

cumplen con ninguno de estos criterios han sido descartados. Las agrupaciones válidas se almacenan en dos tablas diferentes dentro de la base de datos:

- **Tabla apple:** contiene exclusivamente los artículos relacionados directamente con Apple.
- **Tabla entorno:** almacena los artículos que hacen referencia al entorno tecnológico cercano de Apple, según las palabras clave definidas.

Ambas tablas comparten la misma estructura de campos:

- **article_date:** fecha de publicación del artículo (tipo DATETIME)
- **title:** título de la noticia
- **source:** nombre del medio de origen
- **link:** URL directa al artículo

Esta estructura estandarizada no solo facilita la consulta y recuperación de información específica en etapas posteriores del proyecto, sino que también permite una integración sencilla con herramientas de análisis y visualización. Su diseño simple y funcional ha sido clave para garantizar una gestión eficiente de los datos a lo largo del desarrollo del sistema.

5 ANÁLISIS COMPARATIVO DE LIBRERÍAS DE SENTIMENT ANALYSIS Y SELECCIÓN

El apartado del análisis de sentimiento es uno de los más importantes del proyecto. Obtener de forma automática y correctamente clasificada la carga emocional de un titular será un factor determinante para evaluar la calidad del sistema. De ello dependerá, en gran medida, que el proyecto cumpla sus objetivos y que, posteriormente, el modelo predictivo funcione adecuadamente y sea capaz de anticipar con precisión los valores del mercado.

5.1 Librerías consideradas

A la hora de utilizar una librería ya existente para el análisis de sentimiento, se presentan dos opciones claras: **VADER** y **FinBERT**.

- **VADER** es una herramienta basada en diccionario, especialmente útil para textos cortos y directos, como titulares de noticias o publicaciones en redes sociales.
- **FinBERT** es un modelo basado en la arquitectura BERT, entrenado específicamente para analizar textos del ámbito financiero, lo que, en principio, podría sugerir un mejor rendimiento en este contexto.

Aunque ambas librerías podrían parecer igualmente válidas para este proyecto, se realizarán los estudios comparativos necesarios para determinar cuál de ellas ofrece un rendimiento superior y, por tanto, debe ser integrada en el sistema final.

5.2 Metodología para comparar rendimiento

Para poder evaluar correctamente el rendimiento de cada una de las librerías, se empleó la siguiente metodología:

Se programó un script en Python que accede a la base de datos de las noticias y selecciona, de forma aleatoria, 100 titulares de la tabla "apple". Este mismo script crea una nueva tabla en Access llamada Sample, con la misma estructura que la tabla original, pero añadiendo una columna adicional llamada opinionReal, que queda vacía inicialmente.

Una vez creada la tabla, se procedió a rellenar manualmente la columna `opinionReal` con tres posibles valores: "Positive", "Negative" y "Neutral". El criterio empleado para asignar estas etiquetas fue el juicio humano, considerando el punto de vista de un inversor de Apple ante cada titular. Esto resultó especialmente útil teniendo en cuenta que todos los titulares estaban en inglés. Aunque este método no es infalible, representa la forma más sencilla y práctica de construir un conjunto de datos de validación para comprobar el funcionamiento del modelo.

article_date	title	opinionReal
10/11/2023 9:56:49	Worries Mount For Apple Bulls As Stock Takes A Dive, Analysts Eager	Negative
23/09/2023 15:16:54	Daniel Ives Pounds the Table on Apple Stock	Positive
21/03/2020 0:45:06	Customers can't get their iPhones back if they left them at an Apple	Negative
06/04/2024 11:55:05	New Buy Rating for Apple (AAPL), the Technology Giant	Positive

Ilustración 11: Formato de la tabla Sample con opinionReal

Una vez completada la tabla con las opiniones reales, se desarrollaron dos scripts independientes: uno para analizar los titulares con FinBERT y otro con VADER. Ambos scripts toman como entrada la tabla Sample y, al ejecutarse, procesan los títulos almacenados y actualizan la tabla añadiendo los resultados correspondientes.

El script de FinBERT accede a la columna `title`, donde se encuentra el titular de la noticia, y realiza el análisis de sentimiento utilizando este modelo. El resultado de este análisis se almacena en dos nuevas columnas: `top_sentiment_finBert`, que devuelve una de las tres posibles clasificaciones (Positive, Negative o Neutral), y `sentiment_score_finBert`, que devuelve un valor numérico entre 0 y 1 que representa el nivel de confianza del modelo en su clasificación.

top_sentiment_finBert	sentiment_score_finBert
Negative	0,995921611785889
Neutral	0,999995350837708
Neutral	0,974879145622253
Positive	1
Neutral	0,567563951015472
Neutral	0,999628067016602
Positive	0,999999403953552
Positive	1

Ilustración 12: Ejemplo de columnas añadidas por FinBert

Por su parte, el script de VADER funciona de forma muy similar. También toma los titulares como entrada y añade dos columnas a la tabla Sample: `top_sentiment_vader`, que contiene la clasificación del sentimiento en los mismos tres niveles, y `sentiment_score_vader`, que proporciona un valor numérico entre -1 y 1. En este caso, cuanto más cerca esté el valor de -1, más negativa se considera la noticia; cuanto más próximo a 1, más positiva; y si el valor está cercano a 0, se interpreta como neutral.

<code>top_sentiment_vader</code>	<code>sentiment_score_vader</code>
Negative	-0,0772
Neutral	0
Neutral	0
Neutral	0
Negative	-0,4588
Negative	-0,7351
Negative	-0,3818

Ilustración 13: Ejemplo de columnas añadidas por Vader

Gracias a esta metodología, se obtuvieron dos series de resultados comparables sobre el mismo conjunto de noticias, lo que permitió llevar a cabo una evaluación objetiva del rendimiento de ambas herramientas. De esta forma, toda la información —tanto las etiquetas reales como las predicciones generadas por cada modelo y sus respectivos niveles de confianza— queda contenida en una única tabla estructurada, lo que facilita enormemente el análisis comparativo posterior.

<code>article_date</code>	<code>title</code>	<code>opinionReal</code>	<code>top_sentiment_finBert</code>	<code>sentiment_score_finBert</code>	<code>top_sentiment_vader</code>	<code>sentiment_score_vader</code>
10/11/2023 9:56:49	Briefs Mount For Apple Bulls As Stock Takes A Dive, Analysts Eager	Negative	Negative	0,995921611785889	Negative	-0,0772
23/09/2023 15:16:54	Daniel Ives Pounds the Table on Apple Stock	Positive	Neutral	0,999995350837708	Neutral	0
21/03/2020 0:45:06	Customers can't get their iPhones back if they left them at an Apple Store	Negative	Neutral	0,974879145622253	Neutral	0
06/04/2024 11:55:05	New Buy Rating for Apple (AAPL), the Technology Giant	Positive	Positive	1	Neutral	0
19/10/2021 11:10:05	M1 Pro/Max Processor To Be A 'Game Changer', Apple Beating 'Intel	Positive	Neutral	0,567563951015472	Negative	-0,4588
27/11/2019 12:41:54	UPDATE 1-Apple supplier Japan Display to review past earnings after	Negative	Neutral	0,999628067016602	Negative	-0,7351

Ilustración 14: Formato resultante de la tabla final

El siguiente paso consiste en comprobar qué tan bien funcionan estos modelos en la práctica. Para ello, se han programado otros dos scripts que realizan una **matriz de confusión** comparando las etiquetas reales (`opinionReal`) con las predicciones generadas por cada modelo (`top_sentiment_finBert` y `top_sentiment_vader`, respectivamente).

Además de la matriz, se calcularon diversas métricas estándar para evaluar la calidad de clasificación:

Precisión (Precision): mide la proporción de verdaderos positivos sobre el total de predicciones positivas realizadas.

$$Precision = \frac{True\ positives}{True\ Positives + False\ Positives}$$

Exhaustividad (Recall): mide la proporción de verdaderos positivos sobre el total de casos positivos reales.

$$Recall = \frac{True\ positives}{True\ Positives + False\ Negatives}$$

F1-Score: es la media armónica entre precisión y recall, y ofrece una medida equilibrada de ambas.

$$F1 - Score = 2 * \frac{Precision * Recall}{Precision + Recall}$$

Support: representa el número de muestras reales en cada clase, es decir, cuántas veces aparece cada etiqueta (Positive, Negative, Neutral) en los datos reales.

Macro average: Promedia la métrica (precisión, recall, F1) dando el mismo peso a cada clase, sin importar cuántas muestras tenga cada una.

$$Macro\ average = \frac{1}{N} \sum_{i=1}^N Mi$$

Mi: valor de la métrica (precisión, recall o F1) para la clase i

N : Número de clases

Weighted average: Promedia la métrica considerando el número de muestras por clase, dando más peso a las clases más representadas

$$Weighted\ average = \frac{\sum_{i=1}^N Mi * Si}{Si}$$

Mi : Valor de la métrica para la clase i

Si : Número de muestras (support) de la clase i

N : Número total de clases

Por último, la **matriz de confusión** es una herramienta que permite visualizar el rendimiento de un modelo de clasificación, mostrando cómo se distribuyen las predicciones correctas e incorrectas entre las distintas clases. Para una predicción perfecta todos los elementos deberían estar ubicados en la diagonal principal.

Estas métricas permiten obtener una visión completa del rendimiento de cada librería, no solo en términos de aciertos, sino también analizando los errores más comunes.

5.2.1 Resultados: FinBERT vs opinión real

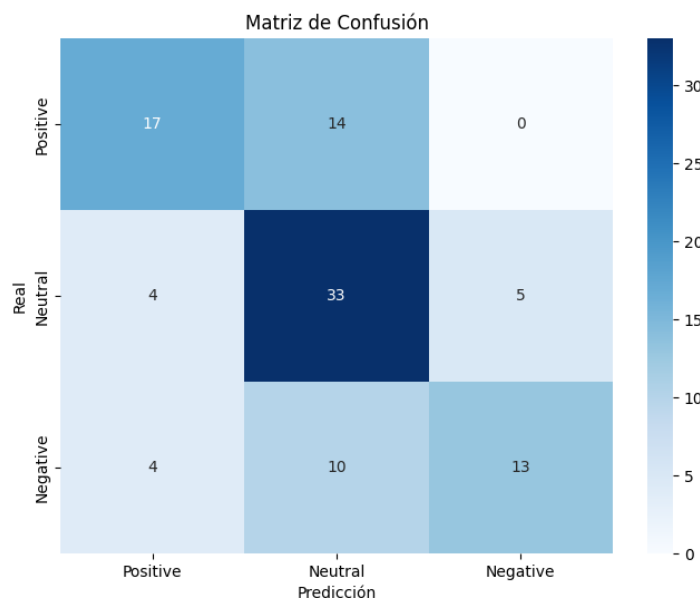


Ilustración 15: Matriz de confusión del FinBERT

Si bien es cierto que en esta primera matriz de confusión la mayoría de los elementos se concentran en la **diagonal principal**, lo cual indica un buen nivel de aciertos por parte del modelo, también se observa un **número no despreciable de errores de clasificación**. La distribución de estos errores es bastante clara: el número de instancias **negativas clasificadas como positivas** es reducido, al igual que el número de **positivas clasificadas como negativas**.

La mayor parte de los errores se concentra en las **celdas adyacentes a la clase "Neutral"**, es decir, en los casos en los que el modelo ha clasificado como **neutrales** titulares que en realidad eran **positivos o negativos**. Este patrón sugiere que el modelo tiende a recurrir a la clasificación neutral cuando la información no es lo suficientemente clara o contundente para inclinarse hacia uno de los extremos.

	precision	recall	F1-score	score
Positive	0.68	0.55	0.61	31
Neutral	0.58	0.79	0.67	42
Negative	0.72	0.48	0.58	27
Accuracy	-	-	0.63	100
Macro avg	0.66	0.61	0.62	100
Weighted avg	0.65	0.63	0.62	100

Tabla 2: Tabla de indicadores de FinBERT

Este patrón se confirma en las métricas de rendimiento del modelo. La clase **"Neutral"** muestra el mejor **F1-score (0.67)** y un alto **recall (0.79)**, lo que indica que el modelo identifica la mayoría de los casos neutrales, aunque con menor precisión. En contraste, las clases **"Positive"** y **"Negative"** tienen una buena precisión (**0.68 y 0.72**), pero un **recall bajo (0.55 y 0.48)**, lo que sugiere que el modelo falla al detectar muchos casos reales de esas categorías.

El **accuracy general es 0.63**, un valor aceptable, pero que refleja la tendencia del modelo a ser conservador, especialmente con los extremos. Para mejorar el rendimiento, convendría centrarse en aumentar la capacidad del modelo para identificar titulares positivos y negativos sin depender tanto de la clase neutral.

5.2.2 Resultados: VADER vs opinión real

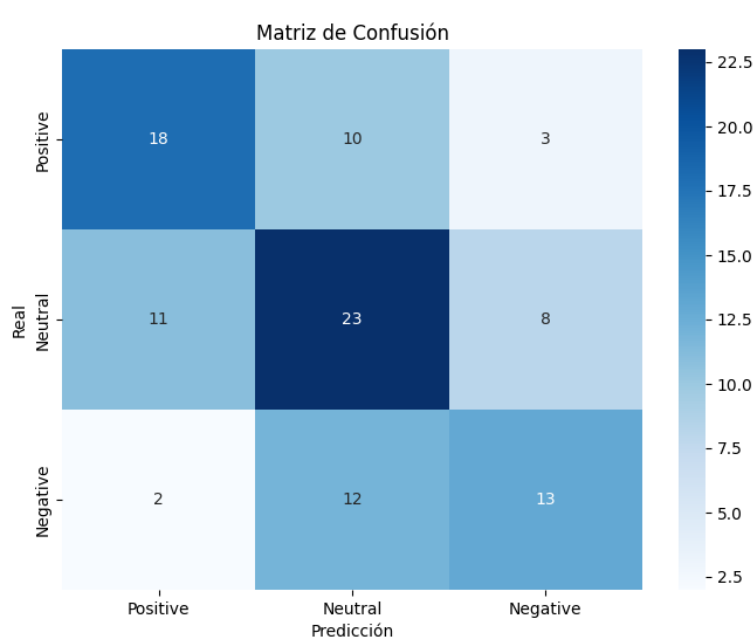


Ilustración 16: Matriz de confusión de VADER

En esta matriz de confusión se observa una distribución de **errores más dispersa** y generalizada en comparación con la anterior. Aunque sigue habiendo una concentración en la diagonal principal —indicativa de aciertos—, el número de clasificaciones **incorrectas** es **considerablemente mayor** y **afecta** a prácticamente **todas las clases**.

A diferencia del caso anterior, donde la mayoría de los errores se concentraban en la clasificación de titulares con polaridad marcada como "Neutral", en esta ocasión los **errores están más repartidos**, lo que sugiere una **mayor confusión del modelo** en todas las direcciones. No solo se confunden positivos o negativos con neutrales, sino que también se observa una mezcla significativa entre clases opuestas.

Este comportamiento refleja una **menor capacidad del modelo para distinguir correctamente entre las diferentes categorías** de sentimiento, lo cual puede afectar negativamente al análisis posterior y, en última instancia, al rendimiento del modelo predictivo.

	precision	recall	F1-score	score
Positive	0.58	0.58	0.58	31
Neutral	0.51	0.55	0.53	42
Negative	0.54	0.48	0.51	27
Accuracy	-	-	0.54	100
Macro avg	0.54	0.54	0.54	100
Weighted avg	0.54	0.54	0.54	100

Tabla 3: Tabla de indicadores de Vader

En comparación con FinBERT, el rendimiento de VADER es **más bajo en todas las métricas**. Las tres clases muestran **F1-scores similares y moderadamente bajos**, sin que ninguna se destaque claramente: **0.58** en *Positive*, **0.53** en *Neutral* y **0.51** en *Negative*. Tanto la **precision** como el **recall** se mantienen en torno al **0.5**, lo que indica que el modelo tiene *dificultades* tanto para *identificar correctamente* las clases como para detectar **todos los casos reales**.

El **accuracy global es de 0.54**, reflejando un desempeño general limitado. A diferencia de FinBERT, aquí no se observa una ventaja clara en la clase Neutral, lo que sugiere que **VADER no logra capturar de forma efectiva los matices del lenguaje financiero** en los titulares analizados. En este contexto, FinBERT demuestra ser más eficaz en la clasificación de sentimientos, especialmente en la clase Neutral.

5.3 Análisis y próximas interpretaciones:

Los resultados de los modelos anteriores, FinBERT y VADER, no han sido especialmente prometedores en términos de precisión general. Por ello, se ha llevado a cabo un análisis más detallado con el objetivo de identificar **posibles causas de las malas clasificaciones**. Una de las hipótesis consideradas es que los modelos podrían estar **tomando decisiones con baja confianza** en los casos en los que se equivocan.

5.3.1 FinBERT – Análisis de confianza media

En el caso de **FinBERT**, la variable `sentiment_score_finBert` en la base de datos indica el nivel de certeza del modelo sobre cada predicción. Este valor se aproxima a 1 cuando el modelo está seguro y desciende conforme la incertidumbre aumenta. Teniendo esto en cuenta, se construyó una **matriz de confusión ponderada**, en la que cada celda representa el **valor medio de `sentiment_score_finBert`** correspondiente a esa combinación específica de predicción y etiqueta real. Esto permite visualizar no solo dónde se equivoca el modelo, sino también **con qué grado de confianza lo hace**.

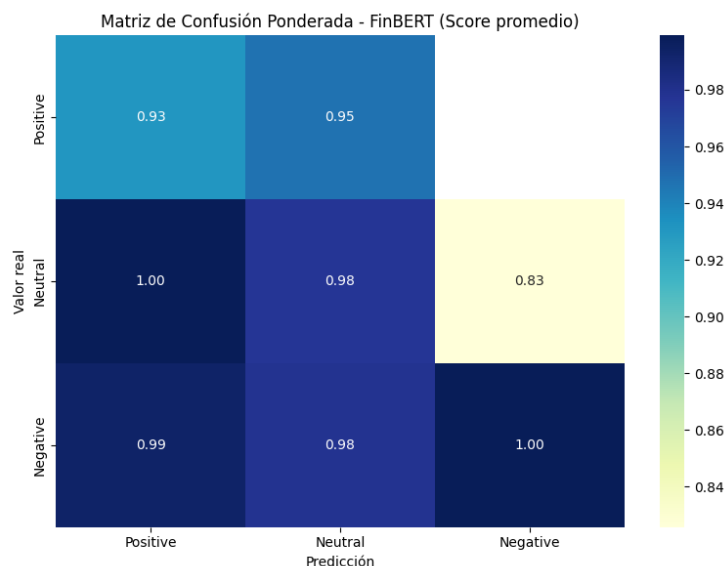


Tabla 4: Matriz de confusión ponderada del FinBERT (score promedio)

Esta primera matriz aclara muchas dudas sobre la confianza del modelo FinBERT en sus decisiones. En principio, los valores de la diagonal principal deberían reflejar una alta seguridad en las predicciones correctas, y en dos de los tres casos esto se cumple. Sin

embargo, llama la atención que la confianza media en los ejemplos correctamente clasificados como **positivos** es **relativamente baja (0.93)** en comparación con otros valores de la matriz, incluyendo varios errores.

De hecho, se observa que en muchas ocasiones el modelo se muestra **más seguro al cometer errores** que al acertar, lo que sugiere que **las equivocaciones no se deben a dudas**, sino a una **falsa certeza**. Este fenómeno es especialmente evidente en las filas *Neutral* y *Negative*, donde las predicciones incorrectas mantienen puntuaciones de confianza muy elevadas (superiores al 0.98).

La única excepción significativa es el caso de los ejemplos **realmente neutrales** que fueron clasificados como **negativos**, donde la confianza media desciende a **0.83**, reflejando cierta indecisión. En conjunto, este análisis evidencia un modelo que **tiende a equivocarse con convicción**, un aspecto preocupante en aplicaciones donde la fiabilidad y la interpretación de la confianza son críticas.

5.3.2 VADER – Análisis de intensidad en errores

De forma análoga, para el modelo **VADER** se ha utilizado la variable `sentiment_score_vader`, que refleja el "compound score" del análisis, también interpretable como una medida de intensidad o seguridad en la clasificación. Se ha generado igualmente una **matriz de confusión basada en la media de estos scores**, con el objetivo de verificar si los errores del modelo están asociados a valores de baja magnitud, lo que sugeriría una clasificación más incierta por parte del sistema.

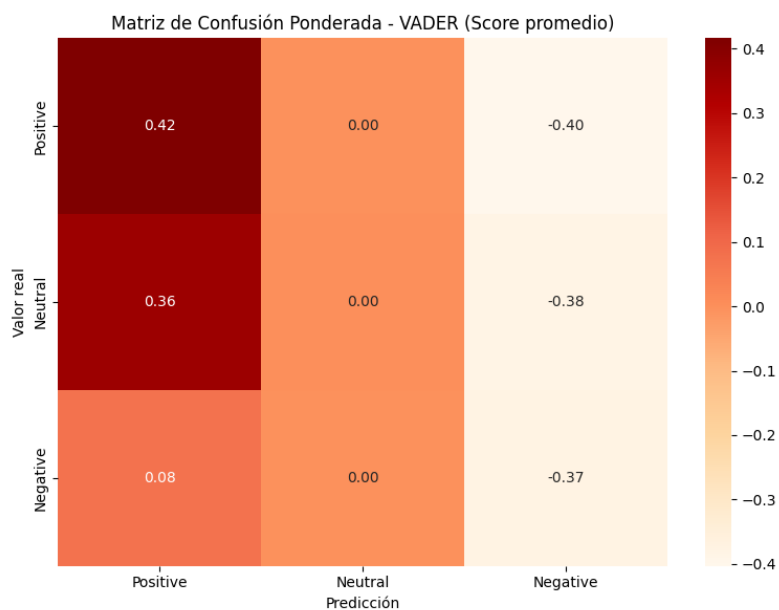


Tabla 5: Matriz de confusión ponderada Vader (score promedio)

En este caso, el problema observado en FinBERT se encuentra aún **más acentuado** en el modelo VADER. La matriz muestra que el modelo asigna valores de confianza **similares** o incluso **más altos a predicciones erróneas** que a las correctas, lo que evidencia una **incapacidad** significativa **para diferenciar** los casos con claridad.

Especialmente llamativo es el caso de los titulares **positivos clasificados erróneamente como negativos**, donde el modelo asigna un **score medio de -0.40**, reflejando una gran certeza en una decisión totalmente incorrecta. Este valor es incluso **más extremo que el score medio de los casos correctamente clasificados como negativos** (que es de -0.37), lo cual es altamente preocupante.

Además, las celdas correspondientes a predicciones **neutrales** muestran sistemáticamente un valor medio de **0.00**, lo que indica que VADER **no diferencia entre clases al clasificar como neutral**, y aplica este resultado incluso en situaciones que claramente no lo son. Este patrón refleja un comportamiento **confuso y poco confiable**, en el que el modelo no solo comete errores, sino que lo hace con un nivel de convicción que dificulta cualquier tipo de control posterior.

En conjunto, esta matriz revela que VADER no solo **falla con frecuencia**, sino que además lo hace **con una confianza mal calibrada**, lo que reduce significativamente su utilidad en contextos donde se necesita fiabilidad en la interpretación de sentimientos.

5.4 Conclusiones y selección

La librería seleccionada ha sido **FinBERT**, principalmente debido a su mejor rendimiento en los diferentes aspectos evaluados. En comparación con otras opciones, FinBERT ha mostrado una mayor capacidad para distinguir correctamente entre las distintas clases de sentimiento, además de obtener mejores resultados en métricas como el **F1-score**, la **precisión** y el **recall**. También se exploró la posibilidad de ajustar el modelo FinBERT mediante *fine-tuning* sobre el conjunto de 100 datos de prueba etiquetados manualmente.

De esta forma, teóricamente, podrían obtenerse mejores resultados y, por lo tanto, mejorar la predicción del modelo. Sin embargo, tras realizar diferentes pruebas, el rendimiento no mejoraba e incluso empeoraba en muchas ocasiones. Esto puede deberse, entre otras razones, a la cantidad reducida de datos etiquetados disponibles.

Aunque la validación se ha realizado sobre una muestra de **100 noticias aleatorias**, lo que representa una proporción limitada del total, se considera que esta muestra es lo suficientemente representativa como para extraer conclusiones preliminares sobre el desempeño general del modelo. Por tanto, y dado su comportamiento más robusto, se ha decidido utilizar FinBERT en las siguientes fases del proyecto.

6 DESARROLLO DEL DATASET

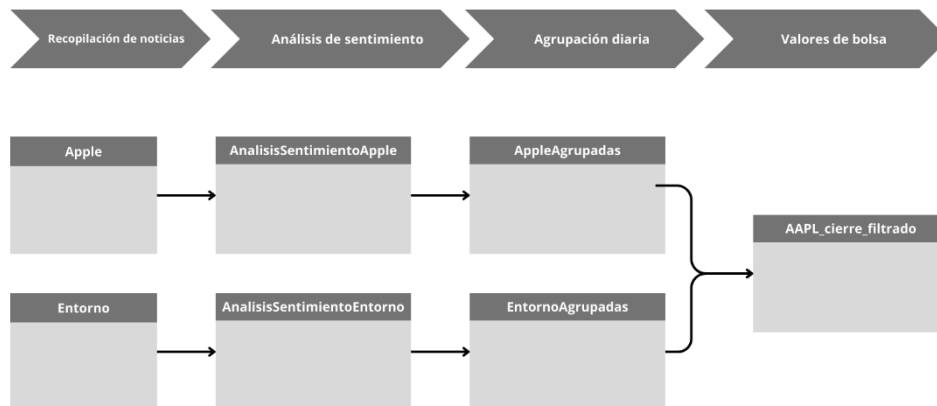


Ilustración 17: Análisis de flujo de datos y estructura del dataset

El apartado de **tratamiento del dataset** tiene como objetivo principal facilitar la comprensión de los pasos que se han seguido durante este Proyecto de Fin de Grado. A través de este apartado, se busca no solo mostrar el flujo de datos y la estructura de las tablas utilizadas en la base de datos, sino también explicar con claridad las decisiones tomadas en cada fase del proceso.

Se describirán de forma ordenada los distintos apartados y etapas del flujo de trabajo, mencionando las variables presentes en cada tabla y, cuando sea necesario, detallando su origen y el método seguido para generarlas o analizarlas. De esta manera, se pretende ofrecer una visión global y justificada del proceso de construcción del dataset, facilitando la interpretación de los resultados y aportando transparencia a la lógica empleada en el desarrollo del sistema.

6.1 Recopilación de noticias

En esta primera fase de recopilación de noticias, tal como se mencionó en el apartado anterior, la base de datos cuenta con dos tablas: **apple** y **entorno (ver anexo 4)**. Ambas tablas comparten exactamente la misma estructura, compuesta por los siguientes campos:

- **Article_date:** Fecha de publicación del artículo
- **Title:** Título de la noticia
- **Source:** Fuente de la información
- **Link:** Link a la noticia

6.2 Análisis de sentimiento

Utilizando la librería de Python **FinBERT**, se emplea un script que toma como entrada las dos tablas mencionadas anteriormente: **apple** y **entorno (ver anexo 3)**. Este script genera como salida dos nuevas tablas: **AnalisisSentimientoApple** y **AnalisisSentimientoEntorno**.

La estructura de ambas tablas de salida es exactamente la misma, y resulta muy similar a la de las tablas de entrada. Los campos que contienen son los siguientes:

- **Article_date:** fecha de publicación del artículo.
- **Title:** título de la noticia.
- **Top_sentiment:** sentimiento general de la noticia, con tres posibles valores: *positive*, *negative* o *neutral*. Este valor corresponde al sentimiento del análisis FinBERT
- **Sentiment_score:** valor numérico entre 0 y 1 que representa el nivel de confianza del modelo respecto al sentimiento asignado.

6.3 Agrupación diaria

Las tablas obtenidas del análisis de sentimiento presentan una estructura en la que algunos días pueden contener numerosas noticias, mientras que en otros no se registra ninguna.

Para facilitar los pasos posteriores del proyecto, se agruparon las noticias por fecha utilizando Python.

Como resultado de este proceso, se generaron dos nuevas tablas en Access: **AppleAgrupadas** y **EntornoAgrupadas (ver anexo 3)**. Estas tablas tienen una estructura relativamente diferente a las anteriores, y están compuestas por los siguientes campos:

- **article_date**: fecha de las noticias agrupadas.
- **Positive**: número de noticias con sentimiento positivo ese día.
- **Negative**: número de noticias con sentimiento negativo ese día.
- **Neutral**: número de noticias con sentimiento neutral ese día.
- **sentimiento_del_dia**: valor numérico calculado a partir del balance entre noticias positivas y negativas, representando el sentimiento agregado del día.

El valor del **sentimiento del día** se ha calculado teniendo en cuenta que, según diversos estudios, las noticias negativas tienen una mayor repercusión en la percepción del lector que las noticias positivas. Considerando este hecho, se ha empleado la siguiente fórmula sencilla para el cálculo del sentimiento diario: (Wang Rakoen Maertens Kai Qin Chan Jon Roozenbeek Sander van der Linden et al., n.d.)

$$\text{sentimiento del día} = N_{pos} * w_{pos} - N_{neg} * w_{neg}$$

N_{pos} : Número de noticias positivas del día

w_{pos} : Peso de las noticias positivas

N_{neg} : Número de noticias negativas del día

w_{neg} : Peso de las noticias negativas

A la hora de definir el peso asignado a las noticias positivas y negativas, se ha optado por utilizar un valor de **1.5** para las positivas y **2.5** para las negativas. Aunque el peso negativo es significativamente mayor, se considera una estimación razonable que refleja adecuadamente el mayor impacto que suelen tener las noticias negativas sobre la percepción del lector y, potencialmente, sobre el comportamiento del mercado.

6.4 Valores de bolsa

Esta última tabla se construye a partir de la importación diaria de los datos bursátiles de la empresa Apple, comenzando desde la fecha del artículo más antiguo disponible entre las noticias recopiladas (**ver anexo 3**). Teniendo esto en cuenta, la tabla final incluye las siguientes variables:

- **Fecha:** fecha correspondiente al registro diario.
- **Close:** precio de cierre de la acción.
- **MediaMovil20:** Media móvil del precio de cierre en una ventana de 20 días. (Ver anexo 2)
- **MediaMovil60:** Media móvil del precio de cierre en una ventana de 60 días.
- **Sentimiento Apple:** valor agregado del sentimiento de las noticias relacionadas directamente con Apple. Se obtienen de la tabla AppleAgrupadas.
- **Sentimiento Entorno:** valor agregado del sentimiento de las noticias sobre el entorno tecnológico. Se obtiene de la tabla EntornoAgrupadas.
- **Opinión General:** combinación ponderada de los dos sentimientos anteriores.
- **Media Móvil 5 días:** media móvil exponencial de 5 días de la opinión general. (Ver anexo 2)
- **Media Móvil 20 días:** media móvil exponencial de 20 días de la opinión general.

Esta tabla se ha generado a partir de un script en Python que asigna, cuando corresponde, el sentimiento asociado a Apple y a su entorno para cada día de cotización. Posteriormente, utilizando una fórmula similar a la empleada para calcular el sentimiento diario por noticia, se calcula el valor de la opinión general. En este caso, las variables tienen pesos diferenciados: las noticias relacionadas directamente con Apple tienen un peso de 1.0, mientras que las del entorno reciben un peso de 0.5.

Esta diferencia de ponderación se debe a que las noticias específicas sobre Apple suelen tener un impacto más directo y significativo en el comportamiento de su acción, mientras que las noticias del entorno tecnológico afectan de forma más indirecta. Al reducir el peso de estas últimas, se busca reflejar con mayor precisión la influencia real de cada tipo de noticia sobre el mercado. (Paul Ryan & Richard Taffler, 2004)

Además, se han calculado medias móviles sobre los valores bursátiles, utilizando ventanas de **20 y 60 días**, que son periodos de tiempo comúnmente empleados en análisis técnico. Estas medias permiten identificar tendencias y posibles oportunidades de inversión, ofreciendo una visión más clara del comportamiento del mercado a corto y medio plazo. (IG, 2025)

En comparación, las medias móviles para la variable de sentimiento se aplican sobre plazos más cortos, tales como 5 y 20 días. Aquí se decidió así usar medias móviles exponenciales (EMA) en vez de las medias simples, ya que el sentimiento es una variable que se ve afectada fuertemente por el reciente movimiento. Las EMA asignan peso superior a los más recientes valores, lo que permite capturar mejor las fluctuaciones rápidas para la comprensión del mercado. Esta es una propiedad que resulta particularmente valiosa para el análisis del sentimiento, cuya fuerza tiende a estar concentrada en plazos temporales cortos.

6.5 Dataset final

Esta última tabla será la que se utilizará para entrenar y validar el modelo de predicción. Las variables seleccionadas pueden o no tener un impacto significativo en la mejora del rendimiento predictivo, un aspecto que será analizado en apartados posteriores. La tabla final cuenta con la siguiente estructura:

- **Fecha:** fecha correspondiente al registro diario.
- **Close:** precio de cierre de la acción.
- **MediaMovil20:** Media móvil del precio de cierre en una ventana de 20 días.
- **MediaMovil60:** Media móvil del precio de cierre en una ventana de 60 días.
- **Media_movil5 (opinión general a 5 días):** media móvil exponencial de 5 días de la opinión general.
- **Media_movil20 (opinión general a 20 días):** media móvil exponencial de 20 días de la opinión general.

Este apartado ha permitido construir un dataset integrado y estructurado, combinando información bursátil y análisis de sentimiento. Cada fase —desde la recopilación de noticias hasta la generación de variables técnicas— ha sido diseñada para asegurar la coherencia y relevancia de los datos.

7 ANÁLISIS, ENTRENAMIENTO Y VALIDACIÓN DEL MODELO

En este apartado se aborda el proceso de entrenamiento y validación del modelo predictivo seleccionado. Para este propósito se ha optado por utilizar una red neuronal LSTM, ya que este tipo de arquitectura está especialmente diseñada para trabajar con series temporales y capturar dependencias a lo largo del tiempo. A diferencia de modelos más simples como la regresión lineal, las LSTM permiten modelar relaciones no lineales y efectos acumulativos, lo que resulta especialmente útil en contextos financieros donde el comportamiento del mercado puede verse influido por patrones históricos y variables de sentimiento.(Kuang, 2023)

7.1 Preparación de los datos: alineación de datos

Antes de proceder al entrenamiento del modelo, es fundamental asegurarse de que todos los datos estén correctamente alineados en el tiempo. Este proyecto trabaja con series temporales que combinan datos bursátiles con información proveniente del análisis de noticias del mercado.

Dado que el objetivo es analizar las predicciones basadas únicamente en información pasada, es imprescindible que los datos estén ordenados cronológicamente. Para garantizar esta coherencia temporal, la tabla ha sido cuidadosamente organizada por fecha, evitando valores nulos y asignando los valores de sentimiento correspondientes a cada momento. Todo este proceso se lleva a cabo con el objetivo de evitar data leakage (incorporación de información futura en el conjunto de entrenamiento), algo especialmente crítico en modelos secuenciales como las redes LSTM.

7.2 División en conjunto de entrenamiento y prueba

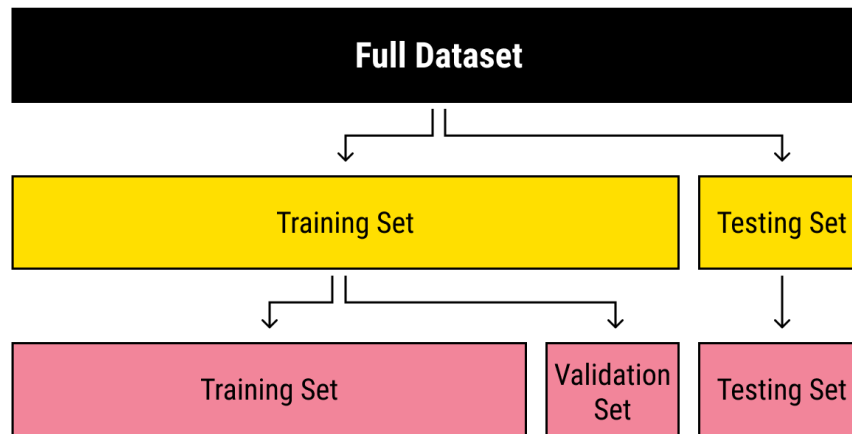


Ilustración 18: División del dataset

Como en cualquier modelo de machine learning, es necesario dividir el conjunto de datos en distintos subconjuntos. Sin embargo, a diferencia de otros casos donde los datos no son secuenciales, en los modelos basados en series temporales no se puede realizar una división aleatoria, ya que esto rompería la relación temporal entre observaciones. En su lugar, la división debe hacerse de forma cronológica para preservar la secuencia y coherencia de los datos.

En este proyecto, el conjunto de datos se divide en tres partes: entrenamiento (80 %), validación (10 %) y prueba (10 %). Esta partición se realiza siguiendo el orden temporal de los datos. El conjunto de entrenamiento se utiliza para ajustar los pesos del modelo, el conjunto de validación sirve para afinar los hiperparámetros y prevenir el sobreajuste (overfitting), y finalmente, el conjunto de prueba se emplea para evaluar el rendimiento del modelo sobre datos completamente nuevos. Este enfoque permite construir un modelo que, con mayor o menor precisión, es capaz de anticipar el comportamiento del mercado a partir de la información histórica disponible.

7.3 Entrenamiento del modelo: Configuración y conjuntos de variables.

El objetivo principal de este proyecto es analizar la relación entre el análisis de sentimiento y los movimientos bursátiles. Teniendo esto en cuenta, se entrenarán dos modelos diferentes, denominados A y B:

El modelo A se entrena utilizando únicamente variables relacionadas con el comportamiento del precio de la acción, concretamente: el precio de cierre, la media móvil a 20 días y la media móvil a 60 días.

El modelo B incluye las mismas variables técnicas que el modelo A, pero además incorpora las **medias móviles del sentimiento**: tanto la media a 5 días como la media a 20 días.

Ambos modelos están basados en redes neuronales LSTM. El objetivo es comprobar si la inclusión de variables sentimentales mejora la capacidad predictiva del modelo respecto al valor de cierre de la acción.

7.4 Ajuste de hiperparámetros

Dentro de las redes neuronales, los **hiperparámetros** son variables que no se aprenden durante la fase de entrenamiento, pero que influyen significativamente en el rendimiento del modelo. A diferencia de los pesos internos de la red, que se ajustan **automáticamente** mediante el proceso de entrenamiento, los **hiperparámetros** deben definirse **previamente**. En otros modelos de *machine learning*, estos parámetros suelen ajustarse mediante validación cruzada (*cross-validation*); sin embargo, esta técnica no es adecuada en redes neuronales LSTM aplicadas a series temporales, ya que el **orden cronológico** de los datos es **fundamental** y proporciona un contexto crucial para el modelo.

Su correcta configuración resulta clave para evitar tanto **el sobreajuste (overfitting)** como **el subajuste (underfitting)**, logrando así un modelo más robusto y preciso. En este proyecto, se han definido los hiperparámetros más comunes en redes neuronales LSTM, que son:

- **Número de épocas (epochs)**: Este representa el número de veces que el modelo recorre el conjunto de entrenamiento.

- **Tamaño del lote (batch size):** número de secuencias (o muestras) procesadas juntas antes de una actualización de pesos.
- **Ventana temporal (window size):** Número de días anteriores usados como entrada para predecir el siguiente valor.

Los valores de los hiperparámetros se han seleccionado mediante pruebas experimentales (ver anexo 1) que comparaban el rendimiento de los modelos y su tiempo de ejecución. Los indicadores más significativos relacionados con la precisión han sido el MAE y el RMSE. Los valores seleccionados para los hiperparámetros son los siguientes:

- **Epochs:80**
- **Batch size:16**
- **Window size:10**

7.5 Evaluación de métricas

Tal y como se ha mencionado anteriormente, se han empleado diferentes métricas no solo para determinar experimentalmente los valores óptimos de los hiperparámetros, sino también para evaluar el rendimiento del modelo seleccionado.

- **Error absoluto medio (MAE):** Esta métrica mide el promedio de los errores absolutos entre las predicciones del modelo y los valores reales. (Ver anexo 2)
- **Raíz del error cuadrático medio (RMSE):** Similar al MAE, pero penaliza más los errores grandes. Es especialmente sensible a valores atípicos (outliers). (Ver anexo 2)
- **Coefficiente de determinación (R^2):** Indica qué proporción de la variabilidad de la variable objetivo (precio de cierre) puede ser explicada por el modelo. Su valor va de 0 a 1, siendo 1 una predicción perfecta. (Ver anexo 2)

8 ANÁLISIS DE RESULTADOS Y DISCUSIÓN

8.1 Interpretación de los resultados obtenidos

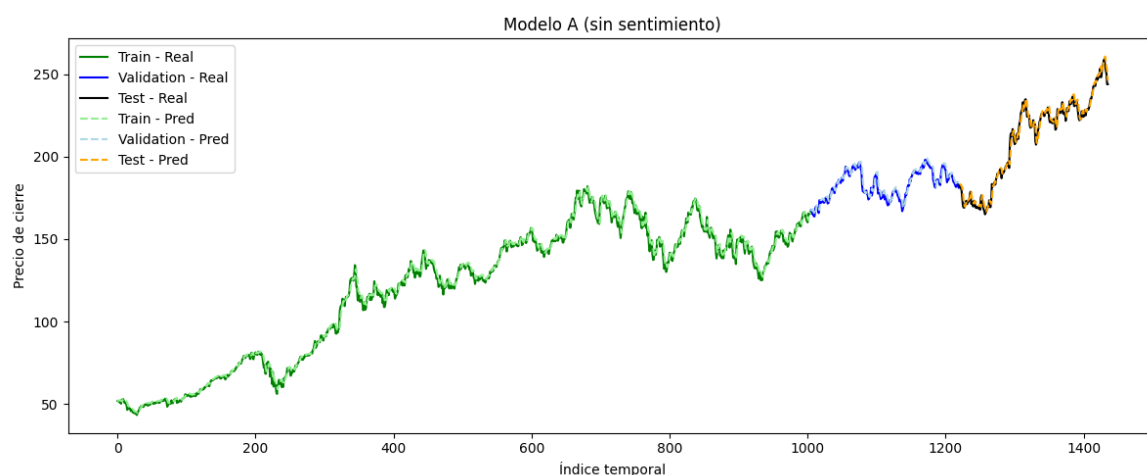


Ilustración 19: Comparación del rendimiento del modelo A (sin sentimiento)

Empezando por el rendimiento del modelo A (sin análisis de sentimiento), gráficamente se observa una correlación y un rendimiento muy buenos a lo largo del periodo analizado. Por tanto, aparentemente, este modelo podría ser una opción adecuada para predecir el comportamiento del mercado.

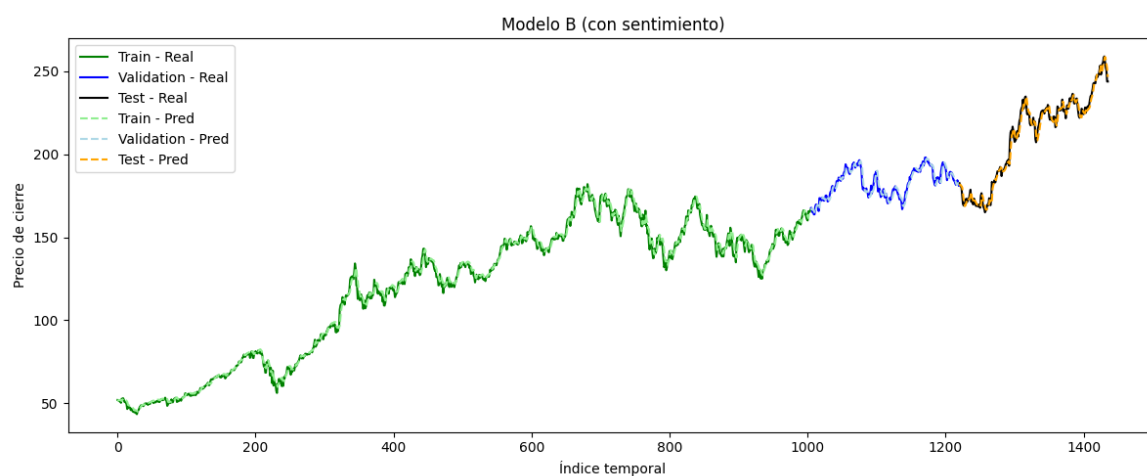


Ilustración 20 Comparación del rendimiento del modelo B (con sentimiento)

El modelo B (con análisis de sentimiento) sigue una línea similar al primer modelo. Aparentemente, se adapta bien a la evolución de la acción y la predice de forma precisa. Sin embargo, es recomendable complementar este análisis visual con los indicadores cuantitativos correspondientes. Estos datos se han obtenido fijando la misma semilla (en este caso la 42), posteriormente se hablará de como cambiar la semilla afecta al resultado.

Modelo	MAE	RMSE	R2
A (sin sentimiento)	2.5048	3.357	0.9833
B (con sentimiento)	2.6285	3.427	0.98607

Tabla 6: Comparación entre modelos

Como conclusión general, puede observarse que el primer modelo (A) presenta un rendimiento global superior al del modelo B en dos de los tres indicadores, siendo el R square el único indicador en el que el modelo B supera al A (cabe recalcar la pequeña diferencia que existe entre ambos). Esto contradice la hipótesis inicial, según la cual añadir el análisis de sentimiento de las noticias mejoraría el rendimiento y la capacidad predictiva del modelo.

Para comprobar la consistencia de esta tendencia, se han empleado diferentes semillas aleatorias (ver anexo 1), y en todos los casos se ha observado el mismo comportamiento: el modelo B resulta ser un predictor menos preciso que el modelo A. Es cierto que la diferencia en los indicadores no es especialmente significativa, pero, aun así, la predicción sigue siendo peor.

Estos resultados podrían sugerir que el proceso ha concluido, pero no es así. Los valores obtenidos en ambos modelos son *sospechosamente* prometedores; es decir, aunque no son perfectos, aparentan captar las tendencias del valor de la acción con una precisión excesiva. Este comportamiento puede observarse gráficamente en los siguientes casos:

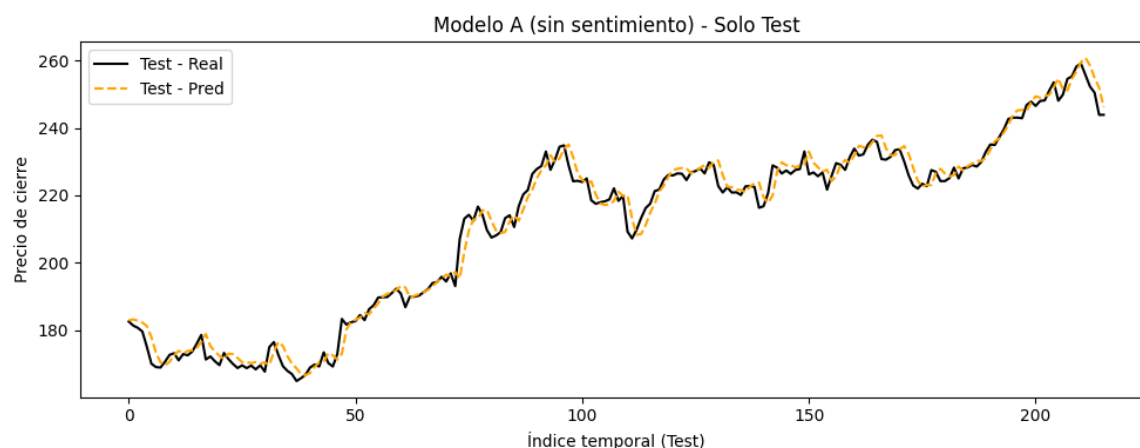


Ilustración 21: Comparación Test valor real vs predicción modelo A (sin sentimiento)

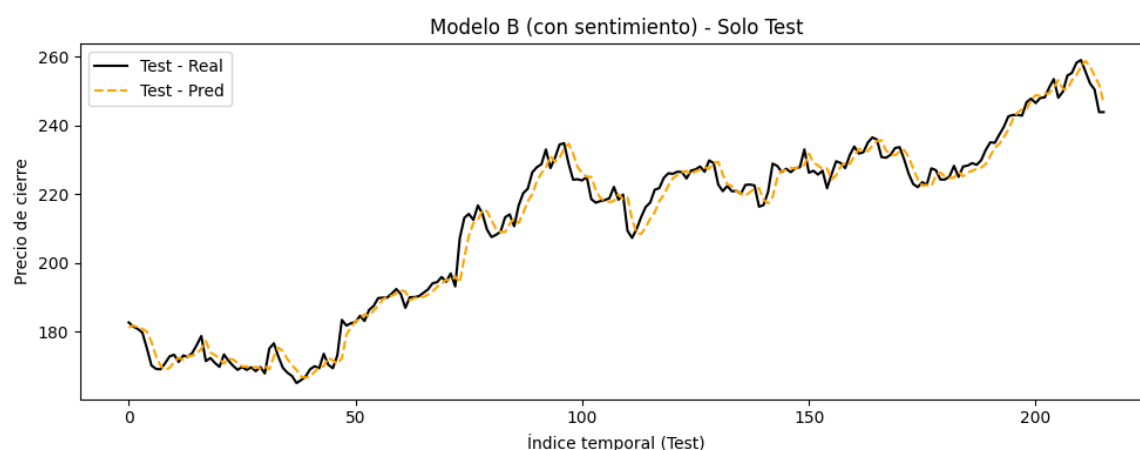


Ilustración 22 Test valor real vs predicción modelo B (con sentimiento)

En ambos gráficos, la curva de la predicción parece replicar el comportamiento del valor real de la acción con un cierto desfase temporal. Es decir, lo que aparentemente podría estar ocurriendo es que el modelo aprende a predecir el valor actual basándose en el comportamiento de uno o dos días anteriores, en lugar de anticipar cambios futuros de manera efectiva.

Esto sugiere que el modelo podría estar incurriendo en un error sistemático, aprendiendo que la mejor forma de minimizar el error es copiar los valores recientes con un pequeño *lag*.

Este tipo de comportamiento se asemeja al de un **modelo de persistencia** (*naïve forecasting*), que simplemente asume que el valor futuro será igual al más reciente. Aunque puede ofrecer buenos resultados visuales, no proporciona una ventaja real en términos de predicción.

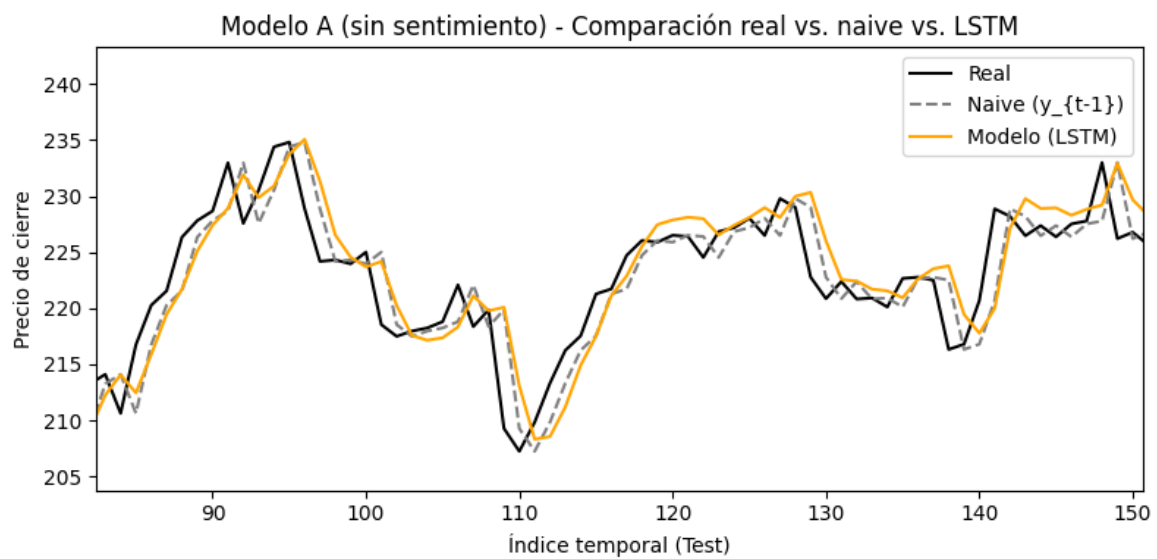


Ilustración 23: Comparación entre real vs naive vs LSTM modelo A

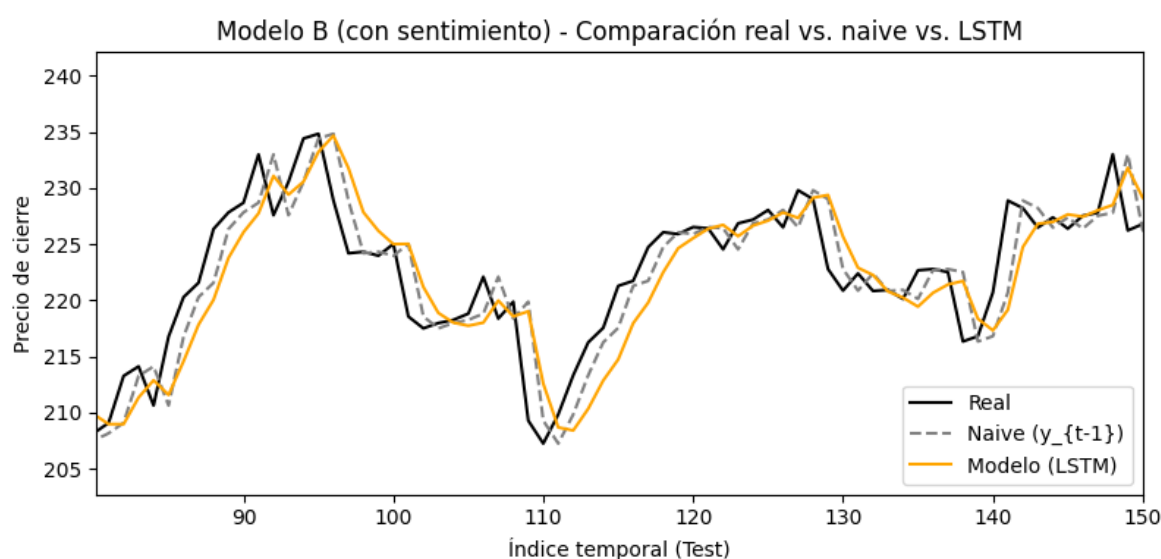


Ilustración 24: Comparación entre real vs naive vs LSTM modelo A

En los gráficos puede observarse que el comportamiento del modelo no es exactamente una copia del día anterior, pero se le asemeja bastante. Podría interpretarse como una reproducción del valor del día anterior con la incorporación de un ruido artificial.

Modelo	MAE	RMSE	R2	MAE Naive
A (42)	2.5048	3.357	0.9833	2.2243
B (42)	2.6285	3.427	0.98607	2.2243

Tabla 7: Tabla de rendimientos de los modelos con MAE Naive

Pese a todo, en la tabla puede observarse que el rendimiento —al menos en términos de MAE— del modelo *Naïve* es superior al de los modelos A y B. Esto podría confirmar la hipótesis de que ambos modelos utilizan, en la práctica, únicamente el precio de cierre como variable predictora, mientras que el resto de la información es interpretada como ruido.

8.2 Análisis de la correlación entre sentimiento y variación de precios

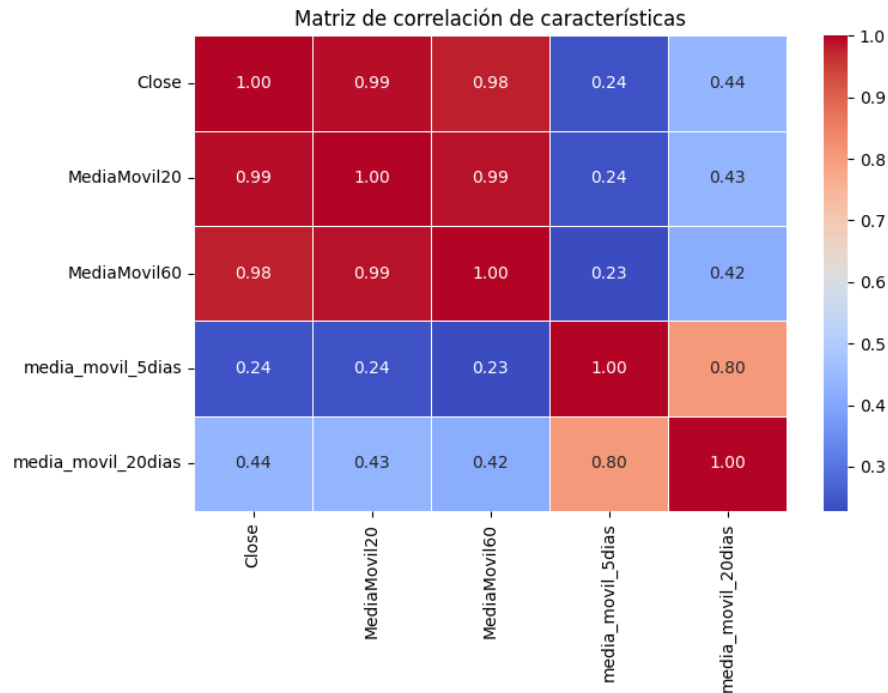


Ilustración 25: Gráfica de correlación entre variables

Teniendo en cuenta las conclusiones expuestas en el apartado anterior, puede intuirse que la relación entre los titulares de noticias y el comportamiento futuro del mercado es muy limitada o incluso inexistente. Si a esta hipótesis le añadimos el gráfico anterior, se refuerza aún más la idea de que, al menos en el contexto de este estudio y con la información disponible, no existe una correlación significativa entre el sentimiento y las variaciones de precio.

En la matriz puede observarse que las variables directamente relacionadas con el precio (como el cierre y las medias móviles) presentan una fuerte correlación entre sí, como era de esperar. Sin embargo, las variables asociadas al sentimiento de las noticias muestran valores de correlación notablemente más bajos, sin ninguna relación destacada con el comportamiento de la acción.

8.3 Limitaciones del estudio y posibles sesgos

Además de las limitaciones ya mencionadas en apartados anteriores, el análisis del modelo permite identificar unas nuevas limitaciones que pueden existir:

- **Sobreajuste:** En este apartado del sobreajuste hay que mencionar que el modelo parece haberse adaptado a predecir patrones a corto plazo o a repetir patrones ya existentes. Una de las razones por las que ha podido ocurrir esto puede ser la poca volatilidad de la acción seleccionada, la falta de cambios significativos en este activo puede haber repercutido en una peor capacidad para predecir el comportamiento futuro.
- **Ruido en el etiquetado:** La falta de correlación entre el precio y el sentimiento del mercado podría estar justificada por una mala capacidad del modelo FinBERT para categorizar titulares ambiguos.

8.4 Pruebas con diferente acción

Teniendo en cuenta lo acontecido durante los últimos años en el sector farmacéutico y su naturaleza completamente distinta al sector tecnológico, se consideró Pfizer como una opción adecuada para evaluar la robustez del modelo. Tomando como referencia los resultados obtenidos al utilizar la acción de Apple, y con la intención de comprobar el comportamiento del modelo con otra acción similar, se optó por una empresa con una mayor volatilidad y perteneciente a un sector diferente, lo que permite analizar su comportamiento en un entorno distinto y determinar si la elección del sector influye en el rendimiento del modelo.

La metodología empleada ha sido exactamente la misma. Utilizando la misma fuente de información, se han recopilado las noticias relacionadas con Pfizer, diferenciando nuevamente entre las que mencionan directamente a la empresa y aquellas que se refieren a su entorno sectorial. Posteriormente, se llevó a cabo un análisis de sentimiento para clasificar las noticias en distintos segmentos. El desarrollo del dataset y su estructura se mantuvieron sin cambios, variando únicamente el nombre de las columnas. Asimismo, se utilizó la misma red neuronal que en el caso anterior, aunque entrenada específicamente con los datos correspondientes a Pfizer.

Utilizando los indicadores previamente definidos, se obtuvo una nueva tabla con los siguientes resultados:

Modelo	MAE	RMSE	R2	MAE Naive
A (42)	0.656	0.983	0.980	0.566
B (42)	0.588	0,857	0.98607	0.566

Tabla 8: Tabla del rendimiento del modelo para Pfizer

Los resultados en este caso difieren de los obtenidos al utilizar las noticias de Apple. El modelo que incorpora los sentimientos se comporta como un mejor predictor que aquel que no los incluye, lo cual se puede comprobar en todos los indicadores evaluados y con las diferentes semillas utilizadas.

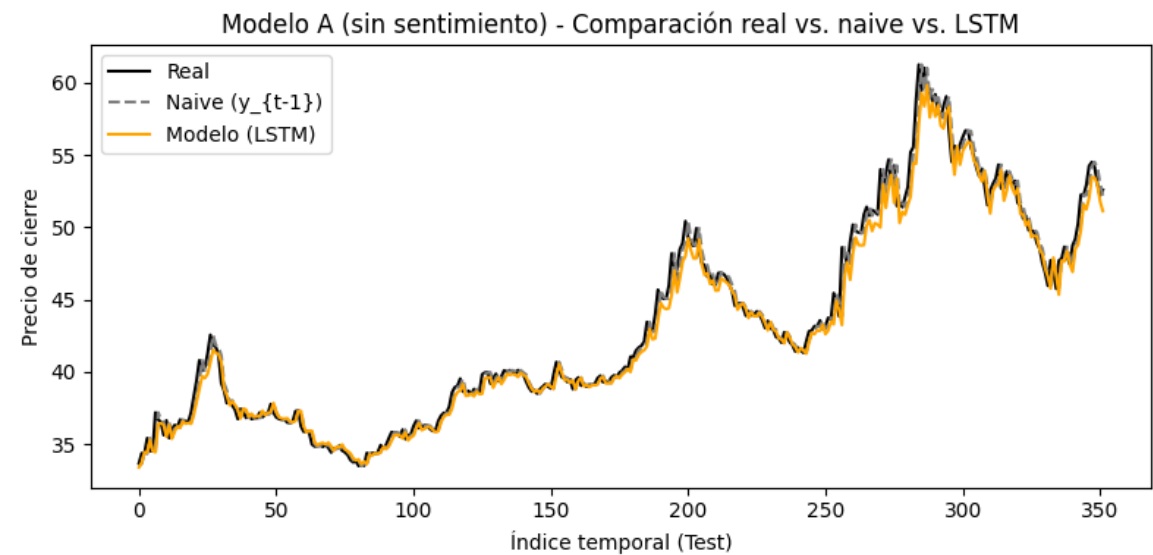


Ilustración 26: Comparación Test valor real vs predicción modelo vs Naive A (sin sentimiento) de Pfizer

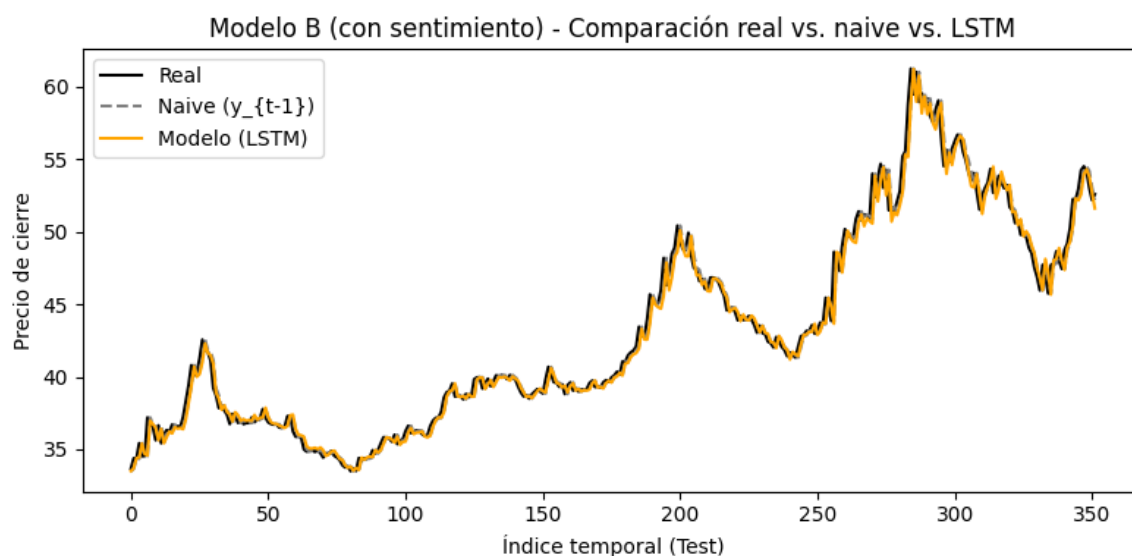


Ilustración 27: Comparación Test valor real vs predicción modelo vs Naive B (sin sentimiento) de Pfizer

Teniendo en cuenta que la fuente de las noticias es la misma y que la metodología aplicada es exactamente la misma, se puede llegar a la siguiente conclusión: el modelo que incorpora los sentimientos ofrece un mejor desempeño predictivo. Además, puede inferirse que la estabilidad y baja volatilidad de la acción de Apple limitan la capacidad predictiva del modelo, en comparación con la acción de Pfizer, esto puede verse también gráficamente (ver anexo 2).

9 CONCLUSIONES Y TRABAJO FUTURO

9.1 Conclusiones generales del proyecto

Con este proyecto se ha conseguido desarrollar un sistema completo en el que se realiza la búsqueda y scraping de noticias financieras, que posteriormente se almacenan en una base de datos. A continuación, dichas noticias se categorizan utilizando librerías de análisis de sentimiento, y esta información se emplea para complementar los indicadores bursátiles clásicos en un modelo de red neuronal LSTM.

Los resultados del proyecto concluyen que, en el caso de Apple, la incorporación de indicadores sentimentales no mejora la calidad de la predicción; de hecho, llega a empeorarla. Sin embargo, en el caso de Pfizer, la precisión del modelo es superior cuando se incluyen estos indicadores, lo que permite concluir que el sector al que pertenece la empresa, el contexto económico y social, así como acontecimientos relevantes como la pandemia de la COVID-19, tienen un peso significativo en la eficacia del modelo.

9.2 Valor añadido y aportaciones personales

Teniendo todo lo anterior en cuenta, se ha demostrado que existe una viabilidad técnica real en la aplicación de modelos de deep learning y procesamiento del lenguaje natural para la predicción bursátil, aunque con las limitaciones inherentes a la calidad de los datos, la disponibilidad de fuentes y las capacidades actuales de las herramientas de análisis de texto.

A título personal, este proyecto ha resultado muy enriquecedor, ya que me ha permitido poner a prueba y consolidar mis conocimientos en áreas como la inteligencia artificial, el *machine learning* y otras disciplinas relacionadas.

Además, ha sido una excelente oportunidad para combinar mis intereses en tecnología y finanzas, lo que me ha llevado a aprender en profundidad sobre el uso de librerías especializadas, la validación de modelos y diversas técnicas de visualización de datos.

9.3 Propuestas de mejora y líneas futuras de investigación

Como mejora futura, sería muy interesante incorporar un mayor número de fuentes de información para realizar el análisis de sentimiento. Además de las noticias tradicionales, podrían utilizarse datos procedentes de redes sociales como Twitter o Reddit, así como informes de analistas y medios especializados como Reuters, Bloomberg, entre otros.

También resultaría valioso ampliar el estudio a más empresas y acciones de distintos sectores, con el objetivo de obtener una perspectiva más general y robusta sobre el impacto del sentimiento en la predicción bursátil.

10 PRESUPUESTO

En todos los proyectos de esta naturaleza se incurren inevitablemente en ciertos gastos, ya sea en forma de licencias de software, herramientas necesarias para el desarrollo, o en las horas de trabajo dedicadas a su ejecución y este proyecto no ha sido la excepción. Para realizar el cálculo del coste de las horas, se ha seleccionado una beca de 1000€ que suele darse para este tipo de proyectos.

Tipo de gasto	Cantidad	€/t	Coste total
Horas dedicadas	300 horas	3,3€/hora	1000€
Licencia ChatGPT	3 meses	23€/mes	69€
Total	-	-	1069€

Tabla: Tabla del presupuesto

11 BIBLIOGRAFÍA

- Billert, F., & Conrad, S. (2024). *A Framework for the Construction of a Sentiment-Driven Performance Index: The Case of DAX40*. <http://arxiv.org/abs/2409.20397>
- Bloomberg. (2023). *Introducing BloombergGPT, Bloomberg's 50-billion parameter large language model, purpose-built from scratch for finance*.
- Cole Stryker. (2024). *¿Qué es una red neuronal recurrente (RNN)?*
- Daniel Lewington, & Laura Sartenaer. (2022). *NLP in Financial Services*.
- Elisabet Furió. (2016). *BBVA Trader: Análisis fundamental vs análisis técnico*.
- Faster Capital. (2025). *Historias De éxito Del Análisis De Sentimiento En Línea*.
- IBM. (2025a). *¿Qué es el análisis de sentimiento?*
- IBM. (2025b). *¿Qué son las redes neuronales?*
- IG. (2025). *Guía para operar con medias móviles*.
- IMARC. (2025). *Informe del mercado de negociación algorítmica por tipo de negociación (divisas (FOREX), mercados de valores, fondos cotizados en bolsa (ETF), bonos, criptodivisas y otros), componentes (soluciones, servicios), modelo de implementación (en las instalaciones, en la nube), tamaño de la organización (pequeñas y medianas empresas, grandes empresas) y región 2024-2032*.
- José Antonio González. (2018). *Hipótesis de Mercado Eficiente*.
- Joseph Nguyen. (2024). *Investing vs. Speculating: What's the Difference?*
- Karanikola, A., Davrazos, G., Liapis, C. M., & Kotsiantis, S. (2023). Financial sentiment analysis: Classic methods vs. deep learning models. *Intelligent Decision Technologies*, 17(4), 893–915. <https://doi.org/10.3233/IDT-230478>
- Kuang, S. (2023). A Comparison of Linear Regression, LSTM model and ARIMA model in Predicting Stock Price A Case Study: HSBC's Stock Price. In *BCP Business & Management FIBA* (Vol. 2023). <https://www.yahoo.com/finance>
- Luis, J., Solivella, R., & Miguel Hernández, U. (2024). *Predicción de Movimientos en los Principales Índices Bursátiles mediante Random Forest*.

- Melissa Horton. (2025). *Maximizing Your Investment Strategy: When to Apply Fundamental, Technical, or Quantitative Analysis*.
- Pablo Huet. (2024). *Técnicas clave para el procesamiento de texto en NLP*.
- Paul Ryan, & Richard Taffler. (2004). *Are Economically Significant Stock Returns and Trading Volumes Driven by Firm-Specific News Releases?*
- Skillcate AI. (2022). *Sentiment Analysis — using NLTK Vader*.
- Talita Greyling, & Stephanié Rossow. (2025). *Twitter sentiment and stock market movements: The predictive power of social media*.
- Wang Rakoen Maertens Kai Qin Chan Jon Roozenbeek Sander van der Linden, D., Note Deming Wang, A., Qin Chan, K., Maertens, R., Roozenbeek, J., van der Linden, S., Author, C., & Wang, D. (n.d.). *NEWS VALENCE AFFECTS NEWS INFLUENCE 1 Is Bad News More Influential Than Good News?*
https://osf.io/3ehzj/?view_only=9dee495d7e2242d9838d418891720c82.
- Wiston Adrián Risso Charquero. (2023). *Procesamiento del Lenguaje Natural en las Finanzas con FinBERT*.
- Yolanda Moro. (2024). *Historia y Evolución de la Bolsa de Valores: El Origen de los Mercados Financieros Modernos*.

ANEXO 1: Tablas informativas

Epochs	Batch Size	Window Size	MAE	RMSE	R2
50	16	10	2.864	3.739	0.979
50	8	10	4.773	5.582	0.953
50	32	10	4.176	5.127	0.961
30	16	10	3.259	4.209	0.973
30	8	10	2.839	3.692	0.978
30	32	10	4.381	5.656	0.952
50	16	20	3.0105	3.951	0.9768
50	16	5	4.415	4.449	0.9707
80	16	10	2.504	3.357	0.9833
80	32	10	2.962	3.966	0.976

Tabla 9: Comparación entre diferentes valores de hiperparametros

Modelo	MAE	RMSE	R2	MAE Naive
A (42)	2.5048	3.357	0.9833	2.2243
B (42)	2.6285	3.427	0.98607	2.2243
A (327)	2.8539	3.6111	0.9807	2.2243
B (327)	3.2891	4.2091	0.9738	2.2243
A (3363)	4.4135	5.279	0.9588	2.2243
B (3363)	3.1741	3.9813	0.9765	2.2243
A (6762)	7.0820	8.0106	0.9050	2.2243
B (6762)	8.3420	9.3186	0.8715	2.2243

A (8445)	4.2206	5.1243	0.9611	2.2243
B (8445)	5.3878	6.464	0.9382	2.2243

Tabla 10: Comparación entre semillas

ANEXO 2. Fórmulas

TF-IDF

$$TF(t, d) = \frac{\text{Número total de veces que el término } t \text{ aparece en el documento } d}{\text{Número total de términos en el documento } d}$$

$$IDF(t, D) = \log\left(\frac{\text{Número total de documentos en el corpus } D}{\text{Número de documentos que contienen el término } t}\right)$$

$$TF - IDF(t, d, D) = TF(t, d) * IDF(t, D)$$

Media móvil

$$\text{Media móvil} = \frac{1}{n} \sum_{i=0}^{n-1} X_{t-i}$$

n : número de periodos

x_{t-i} : es el valor de la variable en el día $t - i$

Medias móvil exponencial

$$EMA_t = \alpha * X_t + 1(1 - \alpha) * EMA_{t-1}$$

EM : Valor actual de la media exponencial

X_t : Valor actual de la serie

$$\alpha = \frac{2}{n + 1} : \text{Factor de suavizado}$$

n : Número de periodos

EMA_{t-1} : EMA del periodo anterior

MAE

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - y_i^b|$$

y_i : valor real

y_i^b : valor predicho

n : número total de observaciones

RMSE

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - y_i^b)^2}$$

y_i : valor real

y_i^b : valor predicho

n : número total de observaciones

R²

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - y_i^b)^2}{\sum_{i=1}^n (y_i - y_i^a)^2}$$

y_i^a : media de los valores reales

ANEXO 3. CODIGO DE PROGRAMACIÓN

A continuación, se incluyen los ficheros principales empleados durante el proyecto.

3.0 Estructura general del código

- Proyecto de fin de grado
 - web_scraper.py
 - analisis_sentimental.py
 - SentimientosAgrupados.py
 - crear_tabla_bolsa.py
 - anadir_sentimiento_bolsa.py
 - redes_neuronales.py
 - pruebas_seed.py
 - Base_de_datos
 - Articulos_sentimiento_bolsa.accdb
 - AAPL.csv

3.1 Web Scraping

Este fichero extrae las noticias de forma automática insertado los campos en una base de datos de Access.

web_scraper.py

```
import os
from bs4 import BeautifulSoup
import requests
import pandas as pd
import pyodbc

# Palabras clave
apple_keywords = ['apple', 'aapl', 'iphone']
entorno_keywords = ['technology', 'tech', 'google', 'microsoft', 'samsung',
                    'amazon', 'intel', 'semiconductor', 'software']

# Crear DataFrames
df_apple = []
df_entorno = []
```

```
# Scrapeo
for page in range(1, 250):
    url = f'https://markets.businessinsider.com/news/aapl-stock?p={page}'
    response = requests.get(url)
    soup = BeautifulSoup(response.text, 'lxml')

    articles = soup.find_all('div', class_="latest-news__story")
    for article in articles:
        article_date = article.find('time', class_='latest-
news__date').get('datetime')
        title = article.find('a', class_='news-link').text.strip()
        source = article.find('span', class_='latest-
news__source').text.strip()
        link = article.find('a', class_='news-link').get('href')
        title_lower = title.lower()

        if any(keyword in title_lower for keyword in apple_keywords):
            df_apple.append((article_date, title, source, link))
        elif any(keyword in title_lower for keyword in entorno_keywords):
            df_entorno.append((article_date, title, source, link))

# Conexión a Access (base de datos real)
script_dir = os.path.dirname(os.path.abspath(__file__))
db_path = os.path.join(script_dir, 'Base_de_datos',
'Articulos_sentimiento_bolsa.accdb')
conn_str = (
    r'DRIVER={Microsoft Access Driver (*.mdb, *.accdb)};'
    rf'DBQ={db_path};'
)
conn = pyodbc.connect(conn_str)
cursor = conn.cursor()

# Crear tabla 'apple' si no existe
if not cursor.tables(table='apple', tableType='TABLE').fetchone():
    cursor.execute("""
        CREATE TABLE apple (
            article_date DATETIME,
            title TEXT,
            source TEXT,
            link TEXT
        )
    """)
    conn.commit()

# Crear tabla 'entorno' si no existe
```

```

if not cursor.tables(table='entorno', tableType='TABLE').fetchone():
    cursor.execute("""
        CREATE TABLE entorno (
            article_date DATETIME,
            title TEXT,
            source TEXT,
            link TEXT
        )
    """)
    conn.commit()

# Insertar en tabla 'apple'
for row in df_apple:
    cursor.execute("""
        INSERT INTO apple (article_date, title, source, link)
        VALUES (?, ?, ?, ?)
    """, *row)

# Insertar en tabla 'entorno'
for row in df_entorno:
    cursor.execute("""
        INSERT INTO entorno (article_date, title, source, link)
        VALUES (?, ?, ?, ?)
    """, *row)

conn.commit()
cursor.close()
conn.close()

print("✅ Noticias clasificadas e insertadas en las tablas 'apple' y 'entorno'.")

```

3.2 Análisis de sentimientos

Utilizando la librería FinBERT se analizan las noticias y los resultados vuelven a incluirse en la base de datos en nuevas tablas.

analisis_sentimental.py

```

import os
import pandas as pd
import pyodbc

```



```
from transformers import AutoTokenizer, AutoModelForSequenceClassification,
pipeline

# Cargar modelo FinBERT
model_name = "yiyanghkust/finbert-tone"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForSequenceClassification.from_pretrained(model_name)
finbert = pipeline("sentiment-analysis", model=model, tokenizer=tokenizer)

# Ruta a la base de datos Access
script_dir = os.path.dirname(os.path.abspath(__file__))
db_path = os.path.join(script_dir, 'Base_de_datos',
                        'Articulos_sentimiento_bolsa.accdb')
conn_str = (
    r'DRIVER={Microsoft Access Driver (*.mdb, *.accdb)};'
    rf'DBQ={db_path};'
)

# Función para analizar sentimiento y guardar en tabla
def analizar_y_guardar(nombre_tabla_origen, nombre_tabla_destino):
    conn = pyodbc.connect(conn_str)
    cursor = conn.cursor()

    # Leer datos de la tabla origen
    query = f"SELECT article_date, title FROM {nombre_tabla_origen}"
    df = pd.read_sql(query, conn)
    df = df.sort_values(by='article_date', ascending=False)

    # Análisis de sentimiento
    sentiments = []
    scores = []

    for title in df['title']:
        result = finbert(title)[0]
        sentiments.append(result['label'])
        scores.append(result['score'])

    df['top_sentiment'] = sentiments
    df['sentiment_score'] = scores

    # Crear tabla destino si no existe
    if not cursor.tables(table=nombre_tabla_destino,
tableType='TABLE').fetchone():
        cursor.execute(f"""
            CREATE TABLE {nombre_tabla_destino} (
                article_date DATETIME,
```

```

        title TEXT,
        top_sentiment TEXT,
        sentiment_score DOUBLE
    )
    """
    conn.commit()

    # Insertar resultados en la tabla destino
    for _, row in df.iterrows():
        cursor.execute(f"""
            INSERT INTO {nombre_tabla_destino} (article_date, title,
            top_sentiment, sentiment_score)
            VALUES (?, ?, ?, ?)
            """, row['article_date'], row['title'], row['top_sentiment'],
            row['sentiment_score'])

    conn.commit()
    cursor.close()
    conn.close()
    print(f"✅ Análisis completado e insertado en la tabla
    '{nombre_tabla_destino}'.")

# Ejecutar para ambas tablas
analizar_y_guardar("apple", "AnálisisSentimientoApple")
analizar_y_guardar("entorno", "AnálisisSentimientoEntorno")

```

3.3 Agrupación de sentimientos

Este fichero agrupa los sentimientos de cada día, teniendo como resultado una tabla más compacta y manejable.

SentimientosAgrupados.py

```

import os
import pandas as pd
import pyodbc

# Ruta a la base de datos Access
script_dir = os.path.dirname(os.path.abspath(__file__))
db_path = os.path.join(script_dir, 'Base_de_datos',
    'Articulos_sentimiento_bolsa.accdb')
conn_str = (

```

```
r'DRIVER={Microsoft Access Driver (*.mdb, *.accdb)};'
rf'DBQ={db_path}';
)

# Función para agrupar y guardar
def agrupar_sentimientos(nombre_tabla_origen, nombre_tabla_destino):
    conn = pyodbc.connect(conn_str)
    cursor = conn.cursor()

    # Leer datos
    df = pd.read_sql(f"SELECT article_date, top_sentiment FROM
{nombre_tabla_origen}", conn)
    df['article_date'] = pd.to_datetime(df['article_date']).dt.date

    # Agrupar por fecha y sentimiento
    grouped = df.groupby(['article_date',
'top_sentiment']).size().unstack(fill_value=0)

    # Asegurar columnas consistentes
    for col in ['Positive', 'Negative', 'Neutral']:
        if col not in grouped.columns:
            grouped[col] = 0

    grouped = grouped[['Positive', 'Negative', 'Neutral']].reset_index()

    # Calcular sentimiento del día
    grouped['sentimiento_del_dia'] = (
        grouped['Positive'] * 1.5 -
        grouped['Negative'] * 2.5
    )

    # Ordenar por fecha ascendente
    grouped = grouped.sort_values(by='article_date')

    # Eliminar la tabla destino si ya existe
    if cursor.tables(table=nombre_tabla_destino,
tableType='TABLE').fetchone():
        cursor.execute(f"DROP TABLE {nombre_tabla_destino}")
        conn.commit()

    # Crear tabla destino sin la columna 'acumulado'
    cursor.execute(f"""
        CREATE TABLE {nombre_tabla_destino} (
            article_date DATE,
            Positive INT,
            Negative INT,
```

```

        Neutral INT,
        sentimiento_del_dia DOUBLE
    )
    """
    conn.commit()

    # Insertar datos
    for _, row in grouped.iterrows():
        cursor.execute(f"""
            INSERT INTO {nombre_tabla_destino} (article_date, Positive,
Negative, Neutral, sentimiento_del_dia)
            VALUES (?, ?, ?, ?, ?)
            """, row['article_date'], int(row['Positive']),
int(row['Negative']), int(row['Neutral']),
            float(row['sentimiento_del_dia']))

    conn.commit()
    cursor.close()
    conn.close()
    print(f"✅ Datos agrupados insertados en '{nombre_tabla_destino}' con
'sentimiento_del_dia'.")

# Ejecutar para ambas tablas
agrupar_sentimientos("AnálisisSentimientoApple", "AppleAgrupadas")
agrupar_sentimientos("AnálisisSentimientoEntorno", "EntornoAgrupadas")

```

3.4 Construcción tabla financiera

Filtra la información de datos bursátiles de un archivo histórico en formato CSV, calculando también las medias móviles a 20 y 60 días almacenando todo esto en una tabla.

crear_tabla_bolsa.py

```

import os
import pyodbc
import pandas as pd

# Ruta al directorio del script
script_dir = os.path.dirname(os.path.abspath(__file__))

# --- BASE DE DATOS ACCESS ---

```

```
db_path = os.path.join(script_dir, 'Base_de_datos',
                        'Articulos_sentimiento_bolsa.accdb')
conn_str = (
    r'DRIVER={Microsoft Access Driver (*.mdb, *.accdb)};'
    rf'DBQ={db_path};'
)

# Consulta para obtener la fecha más antigua
query_fecha = """
    SELECT MIN(article_date) AS fecha_mas_antigua FROM (
        SELECT article_date FROM apple
        UNION ALL
        SELECT article_date FROM entorno
    ) AS todas_las_fechas
"""

try:
    conn = pyodbc.connect(conn_str)
    df_fecha = pd.read_sql(query_fecha, conn)
    conn.close()

    fecha_mas_antigua =
pd.to_datetime(df_fecha.iloc[0]['fecha_mas_antigua']).date()
    print(f"📅 La fecha más antigua encontrada es: {fecha_mas_antigua}")
except Exception as e:
    print(f"❌ Error al acceder a la base de datos: {e}")
    fecha_mas_antigua = None

# --- LECTURA Y FILTRADO DEL CSV ---
csv_path = os.path.join(script_dir, 'Base_de_datos', 'AAPL.csv')

try:
    df_csv = pd.read_csv(csv_path)
    df_csv['Unnamed: 0'] = pd.to_datetime(df_csv['Unnamed: 0'],
errors='coerce').dt.date
    df_csv = df_csv.dropna(subset=['Unnamed: 0'])

    # Obtener la fecha más reciente del CSV
    fecha_mas_reciente = df_csv['Unnamed: 0'].max()
    print(f"📅 La fecha más reciente en el CSV es: {fecha_mas_reciente}")

    # Filtrar por fecha más antigua
    if fecha_mas_antigua:
        df_filtrado = df_csv[df_csv['Unnamed: 0'] >=
fecha_mas_antigua].copy()
        print(f"✅ Datos filtrados desde la fecha más antigua:")
```

```

else:
    print("⚠ No se aplicó el filtro por fecha.")
    df_filtrado = df_csv.copy()

    # Calcular columnas adicionales
    df_filtrado['Diferencial'] = df_filtrado['Close'].pct_change() * 100
    df_filtrado['MediaMovil20'] =
df_filtrado['Close'].rolling(window=20).mean()
    df_filtrado['MediaMovil60'] =
df_filtrado['Close'].rolling(window=60).mean()

    print(df_filtrado[['Unnamed: 0', 'Close', 'Diferencial', 'MediaMovil20',
'MediaMovil60']].head())

    # --- GUARDAR EN ACCESS ---
    conn = pyodbc.connect(conn_str)
    cursor = conn.cursor()
    table_name = "AAPL_cierre_filtrado"

    # Eliminar tabla si existe
    if cursor.tables(table=table_name, tableType='TABLE').fetchone():
        cursor.execute(f"DROP TABLE {table_name}")
        conn.commit()

    # Crear nueva tabla
    cursor.execute(f"""
        CREATE TABLE {table_name} (
            Fecha DATE,
            Close DOUBLE,
            Diferencial DOUBLE,
            MediaMovil20 DOUBLE,
            MediaMovil60 DOUBLE
        )
    """)
    conn.commit()

    # Insertar datos
    for _, row in df_filtrado.iterrows():
        cursor.execute(f"""
            INSERT INTO {table_name} (Fecha, Close, Diferencial,
MediaMovil20, MediaMovil60)
            VALUES (?, ?, ?, ?, ?)
        """,
            row['Unnamed: 0'],
            row['Close'],
            None if pd.isna(row['Diferencial']) else float(row['Diferencial']),

```

```
        None if pd.isna(row['MediaMovil20']) else
float(row['MediaMovil20']),
        None if pd.isna(row['MediaMovil60']) else
float(row['MediaMovil60']))

    conn.commit()
    cursor.close()
    conn.close()
    print(f"✅ Tabla '{table_name}' actualizada con medias móviles de 20 y
60 días.")

except FileNotFoundError:
    print(f"❌ No se encontró el archivo: {csv_path}")
except Exception as e:
    print(f"⚠️ Error al leer o guardar el CSV: {e}")
```

3.5 Integración de los sentimientos

Este fichero une los datos de la tabla bursátil con las tablas tanto de entorno como de la acción concreta resultantes del 4.3.

anadir_sentimiento_bolsa.py

```
import os
import pyodbc
import pandas as pd

# === Configuración de conexión ===
script_dir = os.path.dirname(os.path.abspath(__file__))
db_path = os.path.join(script_dir, 'Base_de_datos',
'Articulos_sentimiento_bolsa.accdb')
conn_str = (
    r'DRIVER={Microsoft Access Driver (*.mdb, *.accdb)};'
    rf'DBQ={db_path};'
)

# Tablas
tabla_cierre = "AAPL_cierre_filtrado"
tabla_sentimiento_apple = "AppleAgrupadas"
tabla_sentimiento_entorno = "EntornoAgrupadas"
```

```
try:
    # == Cargar datos desde Access ==
    conn = pyodbc.connect(conn_str)

    df_cierre = pd.read_sql(f"SELECT * FROM {tabla_cierre}", conn)
    df_cierre['Fecha'] = pd.to_datetime(df_cierre['Fecha']).dt.date

    df_apple = pd.read_sql(f"SELECT article_date, sentimiento_del_dia FROM {tabla_sentimiento_apple}", conn)
    df_apple['article_date'] = pd.to_datetime(df_apple['article_date']).dt.date
    df_apple.rename(columns={'article_date': 'Fecha', 'sentimiento_del_dia': 'Sentimiento_apple'}, inplace=True)

    df_entorno = pd.read_sql(f"SELECT article_date, sentimiento_del_dia FROM {tabla_sentimiento_entorno}", conn)
    df_entorno['article_date'] = pd.to_datetime(df_entorno['article_date']).dt.date
    df_entorno.rename(columns={'article_date': 'Fecha', 'sentimiento_del_dia': 'Sentimiento_entorno'}, inplace=True)

    conn.close()

    # == Fusionar y procesar ==
    df_final = pd.merge(df_cierre, df_apple, on='Fecha', how='left')
    df_final = pd.merge(df_final, df_entorno, on='Fecha', how='left')

    df_final = df_final.sort_values(by='Fecha').reset_index(drop=True)

    # Calcular opinión generalizada
    df_final['Opinion_generalizada'] = df_final['Sentimiento_apple'].fillna(0) + 0.5 * df_final['Sentimiento_entorno'].fillna(0)

    # Medias móviles exponenciales con LAG (shift) aplicado para evitar data leakage
    df_final['media_movil_5dias'] = df_final['Opinion_generalizada'].ewm(span=5, adjust=False).mean().shift(1)
    df_final['media_movil_20dias'] = df_final['Opinion_generalizada'].ewm(span=20, adjust=False).mean().shift(1)

    # Calcular sentimiento acumulado solo con datos reales
    df_final['sentimiento_acumulado'] = df_final['Opinion_generalizada'].where(~df_final['Sentimiento_apple'].isna(), 0).cumsum()
```



```
# Asegurar columnas necesarias
if 'MediaMovil20' not in df_final.columns:
    df_final['MediaMovil20'] = None
if 'MediaMovil60' not in df_final.columns:
    df_final['MediaMovil60'] = None
if 'Diferencial' not in df_final.columns:
    df_final['Diferencial'] = None

# == Escribir nueva tabla Access ==
conn = pyodbc.connect(conn_str)
cursor = conn.cursor()

if cursor.tables(table=tabla_cierre, tableType='TABLE').fetchone():
    cursor.execute(f"DROP TABLE {tabla_cierre}")
    conn.commit()

cursor.execute(f"""
    CREATE TABLE {tabla_cierre} (
        Fecha DATE,
        Close DOUBLE,
        Diferencial DOUBLE,
        MediaMovil20 DOUBLE,
        MediaMovil60 DOUBLE,
        Sentimiento_apple DOUBLE,
        Sentimiento_entorno DOUBLE,
        Opinion_generalizada DOUBLE,
        media_movil_5dias DOUBLE,
        media_movil_20dias DOUBLE,
        sentimiento_acumulado DOUBLE
    )
""")
conn.commit()

for _, row in df_final.iterrows():
    cursor.execute(f"""
        INSERT INTO {tabla_cierre}
        (Fecha, Close, Diferencial, MediaMovil20, MediaMovil60,
         Sentimiento_apple, Sentimiento_entorno, Opinion_generalizada,
         media_movil_5dias, media_movil_20dias, sentimiento_acumulado)
        VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
    """, (
        row['Fecha'],
        row['Close'],
        None if pd.isna(row['Diferencial']) else
        float(row['Diferencial']),
```

```

        None if pd.isna(row['MediaMovil20']) else
float(row['MediaMovil20']),
        None if pd.isna(row['MediaMovil60']) else
float(row['MediaMovil60']),
        None if pd.isna(row['Sentimiento_apple']) else
float(row['Sentimiento_apple']),
        None if pd.isna(row['Sentimiento_entorno']) else
float(row['Sentimiento_entorno']),
        float(row['Opinion_generalizada']),
        None if pd.isna(row['media_movil_5dias']) else
float(row['media_movil_5dias']),
        None if pd.isna(row['media_movil_20dias']) else
float(row['media_movil_20dias']),
        float(row['sentimiento_acumulado'])
    ))

    conn.commit()
    cursor.close()
    conn.close()

    print(f"✅ Tabla '{tabla_cierre}' actualizada correctamente con medias
móviles exponenciales y acumulado corregido.")

except Exception as e:
    print(f"❌ Error en la ejecución: {e}")

```

3.6 Modelo de predicción

Este fichero entrena y evalúa los dos modelos LSTM: uno de ellos con sentimientos y el otro sin. Además de esto, crea visualizaciones gráficas para poder analizar más fácilmente lo que está ocurriendo.

redes_neuronales.py

```

import pandas as pd
import numpy as np
import pyodbc
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_squared_error, mean_absolute_error,
r2_score
import matplotlib.pyplot as plt
from keras.models import Sequential

```

```
from keras.layers import LSTM, Dense
import tensorflow as tf
import random
import os

# === SEMILLA PARA REPRODUCIBILIDAD ===
SEED = 42
np.random.seed(SEED)
random.seed(SEED)
tf.random.set_seed(SEED)
os.environ['PYTHONHASHSEED'] = str(SEED)

# === PARÁMETROS AJUSTABLES ===
EPOCHS = 80
BATCH_SIZE = 16
WINDOW_SIZE = 10

# === CONEXIÓN A BASE DE DATOS ACCESS ===
db_path = r'Base_de_datos\Articulos_sentimiento_bolsa.accdb'
conn_str = (
    r'DRIVER={Microsoft Access Driver (*.mdb, *.accdb)};'
    rf'DBQ={db_path};'
)
conn = pyodbc.connect(conn_str)
df = pd.read_sql("SELECT * FROM AAPL_cierre_filtrado", conn)
conn.close()

# === ORDENAR Y LIMPIAR ===
df = df.sort_values(by='Fecha')
print("\n📊 Filas originales en la base de datos:", len(df))
df = df.dropna()
print("✂ Filas después de eliminar nulos:", len(df))

# === VARIABLES DE LOS DOS MODELOS ===
features_A = ['Close', 'MediaMovil20', 'MediaMovil60']
features_B = features_A + ['media_movil_5dias', 'media_movil_20dias']
target = 'Close'

# === FUNCIÓN PARA PREPARAR DATOS PARA LSTM ===
def prepare_lstm_data(df, features, target, n_steps):
    scaler = MinMaxScaler()
    data = df[features + [target]].copy()
    scaled = scaler.fit_transform(data)
    X, y = [], []
    for i in range(n_steps, len(scaled)):
        X.append(scaled[i - n_steps:i, :-1])
```

```

        y.append(scaled[i, -1])
    return np.array(X), np.array(y), scaler

# === DIVISIÓN PERSONALIZADA ===
def split_data(X, y, train_frac=0.7, val_frac=0.15):
    n = len(X)
    train_end = int(n * train_frac)
    val_end = int(n * (train_frac + val_frac))
    print(f"\nTotal muestras: {n}")
    print(f"Entrenamiento: {train_end}, Validación: {val_end - train_end},
Test: {n - val_end}")
    return (
        X[:train_end], y[:train_end],
        X[train_end:val_end], y[train_end:val_end],
        X[val_end:], y[val_end:]
    )

# === FUNCIONES PARA CONSTRUIR Y EVALUAR EL MODELO ===
def build_lstm_model(input_shape):
    model = Sequential()
    model.add(LSTM(50, activation='relu', input_shape=input_shape))
    model.add(Dense(1))
    model.compile(optimizer='adam', loss='mse')
    return model

def inverse_transform_predictions(scaler, y_pred, y_real):
    n_total_features = scaler.n_features_in_
    pad_pred = np.zeros((len(y_pred), n_total_features))
    pad_true = np.zeros((len(y_real), n_total_features))
    pad_pred[:, -1] = y_pred.flatten()
    pad_true[:, -1] = y_real.flatten()
    inverse_pred = scaler.inverse_transform(pad_pred)[:, -1]
    inverse_true = scaler.inverse_transform(pad_true)[:, -1]
    return inverse_true, inverse_pred

def plot_predictions(inverse_train, inverse_val, inverse_test,
inverse_train_pred, inverse_val_pred, inverse_test_pred, title=""):
    total_len = len(inverse_train) + len(inverse_val) + len(inverse_test)
    x_train = np.arange(0, len(inverse_train))
    x_val = np.arange(len(inverse_train), len(inverse_train) +
len(inverse_val))
    x_test = np.arange(len(inverse_train) + len(inverse_val), total_len)

    plt.figure(figsize=(12, 5))
    plt.plot(x_train, inverse_train, label="Train - Real", color="green")
    plt.plot(x_val, inverse_val, label="Validation - Real", color="blue")

```

```
plt.plot(x_test, inverse_test, label="Test - Real", color="black")

plt.plot(x_train, inverse_train_pred, '--', label="Train - Pred",
color="lightgreen")
plt.plot(x_val, inverse_val_pred, '--', label="Validation - Pred",
color="lightblue")
plt.plot(x_test, inverse_test_pred, '--', label="Test - Pred",
color="orange")

plt.title(title)
plt.xlabel("Índice temporal")
plt.ylabel("Precio de cierre")
plt.legend()
plt.tight_layout()
plt.show()

def plot_test_only(inverse_test, inverse_test_pred, title=""):
    plt.figure(figsize=(10, 4))
    plt.plot(inverse_test, label="Test - Real", color="black")
    plt.plot(inverse_test_pred, '--', label="Test - Pred", color="orange")
    plt.title(f"{title} - Solo Test")
    plt.xlabel("Índice temporal (Test)")
    plt.ylabel("Precio de cierre")
    plt.legend()
    plt.tight_layout()
    plt.show()

def train_and_evaluate_model(name, X_train, y_train, X_val, y_val, X_test,
y_test, scaler):
    print(f"\nEntrenando {name}...")
    model = build_lstm_model((X_train.shape[1], X_train.shape[2]))
    model.fit(X_train, y_train, validation_data=(X_val, y_val),
epochs=EPOCHS, batch_size=BATCH_SIZE, verbose=0)

    y_train_pred = model.predict(X_train)
    y_val_pred = model.predict(X_val)
    y_test_pred = model.predict(X_test)

    inv_train, inv_train_pred = inverse_transform_predictions(scaler,
y_train_pred, y_train)
    inv_val, inv_val_pred = inverse_transform_predictions(scaler,
y_val_pred, y_val)
    inv_test, inv_test_pred = inverse_transform_predictions(scaler,
y_test_pred, y_test)

    print(f"\n📊 Resultados de {name}:")
```

```

print("MAE:", mean_absolute_error(inv_test, inv_test_pred))
print("RMSE:", np.sqrt(mean_squared_error(inv_test, inv_test_pred)))
print("R2:", r2_score(inv_test, inv_test_pred))

# === Gráfico general (train, val, test) ===
plot_predictions(inv_train, inv_val, inv_test, inv_train_pred,
inv_val_pred, inv_test_pred, title=name)

# === Gráfico solo del test ===
plot_test_only(inv_test, inv_test_pred, title=name)

# === Comparación con modelo Naive ( $y_t = y_{t-1}$ ) ===
inv_test_naive = inv_test[:-1]          #  $y_{t-1}$ 
inv_test_real = inv_test[1:]           #  $y_t$ 
inv_test_pred_shifted = inv_test_pred[1:] # LSTM predicción
correspondiente

plt.figure(figsize=(8, 4))
plt.plot(inv_test_real, label="Real", color="black")
plt.plot(inv_test_naive, '--', label="Naive ( $y_{t-1}$ )", color="gray")
plt.plot(inv_test_pred_shifted, label="Modelo (LSTM)", color="orange")
plt.title(f"{name} - Comparación real vs. naive vs. LSTM")
plt.xlabel("Índice temporal (Test)")
plt.ylabel("Precio de cierre")
plt.legend()
plt.tight_layout()
plt.show()

# === MAE del modelo naive ===
naive_mae = mean_absolute_error(inv_test_real, inv_test_naive)
print(f"📏 MAE modelo naive ( $y_t = y_{t-1}$ ): {naive_mae:.4f}")

# === ENTRENAMIENTO Y EVALUACIÓN ===
X_a, y_a, scaler_a = prepare_lstm_data(df, features_A, target, WINDOW_SIZE)
X_b, y_b, scaler_b = prepare_lstm_data(df, features_B, target, WINDOW_SIZE)

X_a_train, y_a_train, X_a_val, y_a_val, X_a_test, y_a_test = split_data(X_a,
y_a)
X_b_train, y_b_train, X_b_val, y_b_val, X_b_test, y_b_test = split_data(X_b,
y_b)

train_and_evaluate_model("Modelo A (sin sentimiento)", X_a_train, y_a_train,
X_a_val, y_a_val, X_a_test, y_a_test, scaler_a)
train_and_evaluate_model("Modelo B (con sentimiento)", X_b_train, y_b_train,
X_b_val, y_b_val, X_b_test, y_b_test, scaler_b)

```

3.7 Evaluación de semillas

Ejecuta múltiples entrenamientos con diferentes semillas y compara rendimientos.

pruebas_seed.py

```
import pandas as pd
import numpy as np
import pyodbc
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_squared_error, mean_absolute_error,
r2_score
from keras.models import Sequential
from keras.layers import LSTM, Dense
import tensorflow as tf
import random
import os

# === PARÁMETROS AJUSTABLES ===
EPOCHS = 80
BATCH_SIZE = 16
WINDOW_SIZE = 10
SEEDS = [42, 327, 3363, 6762, 8445]

# === FUNCIONES AUXILIARES ===
def set_seed(seed):
    np.random.seed(seed)
    random.seed(seed)
    tf.random.set_seed(seed)
    os.environ['PYTHONHASHSEED'] = str(seed)

def prepare_lstm_data(df, features, target, n_steps):
    scaler = MinMaxScaler()
    data = df[features + [target]].copy()
    scaled = scaler.fit_transform(data)
    X, y = [], []
    for i in range(n_steps, len(scaled)):
        X.append(scaled[i - n_steps:i, :-1])
        y.append(scaled[i, -1])
    return np.array(X), np.array(y), scaler

def split_data(X, y, train_frac=0.7, val_frac=0.15):
```

```

n = len(X)
train_end = int(n * train_frac)
val_end = int(n * (train_frac + val_frac))
return (
    X[:train_end], y[:train_end],
    X[train_end:val_end], y[train_end:val_end],
    X[val_end:], y[val_end:]
)

def build_lstm_model(input_shape):
    model = Sequential()
    model.add(LSTM(50, activation='relu', input_shape=input_shape))
    model.add(Dense(1))
    model.compile(optimizer='adam', loss='mse')
    return model

def inverse_transform_predictions(scaler, y_pred, y_real):
    n_total_features = scaler.n_features_in_
    pad_pred = np.zeros((len(y_pred), n_total_features))
    pad_true = np.zeros((len(y_real), n_total_features))
    pad_pred[:, -1] = y_pred.flatten()
    pad_true[:, -1] = y_real.flatten()
    inverse_pred = scaler.inverse_transform(pad_pred)[:, -1]
    inverse_true = scaler.inverse_transform(pad_true)[:, -1]
    return inverse_true, inverse_pred

def evaluate_model(X_train, y_train, X_val, y_val, X_test, y_test, scaler):
    model = build_lstm_model((X_train.shape[1], X_train.shape[2]))
    model.fit(X_train, y_train, validation_data=(X_val, y_val),
epochs=EPOCHS, batch_size=BATCH_SIZE, verbose=0)

    y_test_pred = model.predict(X_test)
    inv_test, inv_test_pred = inverse_transform_predictions(scaler,
y_test_pred, y_test)

    # Cálculo naive (y_t = y_{t-1})
    inv_test_naive = inv_test[:-1]
    inv_test_real = inv_test[1:]
    inv_test_pred_shifted = inv_test_pred[1:]
    naive_mae = mean_absolute_error(inv_test_real, inv_test_naive)

    mae = mean_absolute_error(inv_test, inv_test_pred)
    rmse = np.sqrt(mean_squared_error(inv_test, inv_test_pred))
    r2 = r2_score(inv_test, inv_test_pred)
    return mae, rmse, r2, naive_mae

```



```
# === CONEXIÓN Y PREPARACIÓN DE DATOS ===
db_path = r'Base_de_datos\Articulos_sentimiento_bolsa.accdb'
conn_str = (
    r'DRIVER={Microsoft Access Driver (*.mdb, *.accdb)};'
    rf'DBQ={db_path};'
)
conn = pyodbc.connect(conn_str)
df = pd.read_sql("SELECT * FROM AAPL_cierre_filtrado", conn)
conn.close()

df = df.sort_values(by='Fecha').dropna()

features_A = ['Close', 'MediaMovil20', 'MediaMovil60']
features_B = features_A + ['media_movil_5dias', 'media_movil_20dias']
target = 'Close'

# === BUCLE PRINCIPAL ===
results = []

for seed in SEEDS:
    print(f"\n🔄 Ejecutando con semilla: {seed}")
    set_seed(seed)

    # Modelo A
    X_a, y_a, scaler_a = prepare_lstm_data(df, features_A, target,
WINDOW_SIZE)
    X_a_train, y_a_train, X_a_val, y_a_val, X_a_test, y_a_test =
split_data(X_a, y_a)
    mae_a, rmse_a, r2_a, naive_a = evaluate_model(X_a_train, y_a_train,
X_a_val, y_a_val, X_a_test, y_a_test, scaler_a)

    # Modelo B
    X_b, y_b, scaler_b = prepare_lstm_data(df, features_B, target,
WINDOW_SIZE)
    X_b_train, y_b_train, X_b_val, y_b_val, X_b_test, y_b_test =
split_data(X_b, y_b)
    mae_b, rmse_b, r2_b, naive_b = evaluate_model(X_b_train, y_b_train,
X_b_val, y_b_val, X_b_test, y_b_test, scaler_b)

    results.append({
        "Seed": seed, "Modelo": "A (sin sentimiento)",
        "MAE": mae_a, "RMSE": rmse_a, "R2": r2_a, "MAE_naive": naive_a
    })
    results.append({
        "Seed": seed, "Modelo": "B (con sentimiento)",
        "MAE": mae_b, "RMSE": rmse_b, "R2": r2_b, "MAE_naive": naive_b
```

```
    })

# === MOSTRAR RESULTADOS EN TABLA ===
results_df = pd.DataFrame(results)
print("\n📋 Resultados con métricas extendidas:\n")
print(results_df.pivot(index='Seed', columns='Modelo', values=['MAE',
'RMSE', 'R2', 'MAE_naive']).round(4))
```